SD6103 Group Assignment

HDB Resale Prices and Hawker-Centre Proximity

Tasks 1–4

Team: Mission Number CLW

Members:

G2606270J	WANG YAHUI
G2610762A	CAI SONGLIN
G2610151E	WANG YUANZUO
G2606195K	WANG YUHAO
G2610637A	WANG WEICHENG
G2608301F	LIN YUNFAN

This report develops a lecture-style ER model, implements and populates a normalized PostgreSQL database, answers six analytical queries, and evaluates query runtime as data volume and indexes change. The final database contains 986,771 resale records. Every schema, query and index statement is reproduced in the report (Appendix A, Section 3.1 and Section 4.1); the executable SQL files, transformation programs, raw measurements and plans accompany it as supplementary material.

The project source code is available in our GitHub repository.

Contents

1. Conceptual ER Model and Logical Schema **2**
- 1.1. ER Model . . . 2
- 1.2. Logical Schema Design and Implementation . . . 6

2. Data Acquisition and Population **12**
- 2.1. Data Acquisition . . . 12
- 2.2. Database Population . . . 15

3. SQL Queries and Initial Performance Results **18**
- 3.1. SQL Queries . . . 18
- 3.2. Performance Measurements . . . 27

4. Indices and Their Effect on Runtime **32**
- 4.1. Index Creation . . . 32
- 4.2. Effect on Runtime . . . 33

Declaration of AI Use **37**

A. Schema Creation Commands **39**

1. Conceptual ER Model and Logical Schema

The database links HDB resale transactions to hawker centres for the five analyses in Appendix A of the assignment. Six entity types and one many-to-many proximity relationship map to seven stored relations. The design uses the lecture ER notation and satisfies Boyce–Codd Normal Form (BCNF) under the stated identity and snapshot assumptions. The complete PostgreSQL DDL is listed in Appendix A and supplied as the executable file `sql/01_Create_Database.sql`. [1, 2, 3, 4]

1.1. ER Model

1.1.1. Entities and modelling assumptions

The required entity types are Town, HDB Flat, Resale Transaction and Hawker Centre. HDB Block stores the address and coordinates shared by flat profiles. Source File identifies each input snapshot and its reporting basis.

The public resale data provides a block, street, flat type, model, floor area, storey band and lease commencement year, but no unit number or persistent physical-flat identifier. We therefore model an HDB Flat as an *observed attribute profile*. Its natural identity consists of the block, flat type, model, floor area, lower and upper storey bounds, and lease commencement year. Several physical dwellings can share one profile. This design supports transaction-level analysis but cannot identify repeat sales of a particular dwelling or count unique physical units. [5]

Each published resale record has its own transaction identifier. The source-file identifier and data-record number form an additional candidate key. Identical-looking rows at different source positions are retained; price, month and profile do not establish a unique sale. Each source period contributes one frozen snapshot. Appending overlapping revisions would require a separate reconciliation process.

A block is identified by its canonical block number and street name and belongs to exactly one HDB town in the analytical snapshot. Block numbers remain text because they can contain letters. The town comes from the resale data; a planning area in the geolocation data is not automatically treated as the same classification. Block coordinates are taken from the specified HDB geolocation file. [7]

The NEA catalogue is the authority for centre identity, type and stall counts. Its inspected snapshot contains 107 records: 27 hawker centres (HC), 77 markets with hawker centres (MHC) and 3 markets only (MK). All are retained, but the proximity analyses in Q4 and Q5 use only HC and MHC. A fresh-market hawker additionally has a positive number of market-produce stalls. [6]

The supplied `hawker_geo.csv` has 129 records and supplies coordinates for enrichment. Reliable matching is required; extra geolocation records do not automatically become catalogue centres with invented stall counts. Unmatched catalogue centres may retain missing coordinates. Names and addresses are canonicalized consistently before loading, and conflicting matches require review.

Hawker locations and attributes describe the available snapshot, not an operating history. Comparing them with historical resale prices is descriptive and does not establish that a centre operated at its present location at the time of a sale.

1.1.2. Proximity definition and classification

Proximity is measured by great-circle distance between a block coordinate and a hawker-centre coordinate. All flat profiles in a block use that block's location. Coordinates are interpreted as WGS84 latitude and longitude. With angles in radians and Earth radius $R = 6{,}371{,}000$ metres, the Haversine calculation is

$$
\begin{aligned}
a &= \sin^2\!\left(\frac{\phi_2 - \phi_1}{2}\right) + \cos\phi_1 \cos\phi_2 \sin^2\!\left(\frac{\lambda_2 - \lambda_1}{2}\right), \\
\bar{a} &= \min\{1, \max\{0, a\}\}, \\
d &= 2R \operatorname{atan2}\!\left(\sqrt{\bar{a}}, \sqrt{1 - \bar{a}}\right).
\end{aligned}
$$

The clamp handles floating-point round-off. This is a reproducible geographic measure using the supplied data; it is not a walking distance or travel time.

Class	Distance in metres	Meaning
NEAR	$0 \le d < 500$	Closest access category
MEDIUM	$500 \le d < 1000$	Intermediate access category
FAR	$d \ge 1000$	Furthest access category

These are project-defined thresholds, not official accessibility standards. They are mutually exclusive and exhaustive for valid distances. Exactly 500 metres is MEDIUM and exactly 1000 metres is FAR. Classification uses the unrounded distance. A missing distance remains unknown and is excluded from class-specific averages; it is never assigned to FAR.

The Proximity relationship records each geocoded block–centre pair. The intended population includes all such pairs, including those beyond one kilometre, so the FAR class remains observable. Its conceptual attributes are `distance_m` and `proximity_class`, with the latter determined by the distance rule. At implementation level, distance is stored once per pair and class is computed in a view.

For Q4, the class of a flat is the class of its nearest eligible hawker centre. The same block-level classification applies to every profile and transaction in that block. Equal distances are resolved using the smaller centre identifier. A block 300 metres from one centre and 1400 metres from another is classified as NEAR and does not also enter the FAR group. Each classified sale therefore contributes once to the average for its town and flat type.

For Q5, candidates are first restricted to eligible centres with fresh-market stalls, and the nearest centre is then selected from this subset. The same three distance intervals apply. For example, a block with a general hawker 200 metres away and a qualifying market-hawker 800 metres away is NEAR for Q4 and MEDIUM for Q5. Filtering the general nearest centre after selection would incorrectly exclude this block.

Both classifications mean nearest among reliably geocoded eligible centres in the chosen catalogue. Missing centre coordinates can overstate distance to the true nearest centre. Matching and geocoding coverage must therefore be reported during data population. The budget for Q5 is chosen from the populated data so that the selected towns form a nonempty proper subset.

1.1.3. ER diagram and query support

The complete model appears in Figure 1. Rectangles denote entity types, diamonds denote binary relationships, and separate ovals denote attributes. Underlining marks the primary key. Each oval has one connector, and each diamond connects exactly two entities, following the lecture syntax. [2, 3]

The $(\min, \max)$ label beside an entity describes the participation of one instance of that entity. An asterisk means no fixed upper bound. Thus $(1, 1)$ means exactly one, $(1, *)$ means one or more, and $(0, *)$ means zero or more.

Relationship	**First entity**	**Second entity**
Town comprises HDB Block	Town $(1, *)$	Block $(1, 1)$
HDB Block houses HDB Flat	Block $(1, *)$	Flat $(1, 1)$
HDB Flat describes Resale Transaction	Flat $(1, *)$	Transaction $(1, 1)$
Source File supplies Resale Transaction	Source $(0, *)$	Transaction $(1, 1)$
HDB Block has Proximity to Hawker Centre	Block $(0, *)$	Centre $(0, *)$

Only towns, blocks and profiles represented in resale records enter this snapshot, so each has at least one child in the completed dataset. Foreign keys enforce each child's parent reference but cannot require parents to have children; deriving parents from the resale records establishes these $(1, *)$ minimums. Source File may represent an empty input snapshot, allowing zero transactions. Missing coordinates permit zero proximity records; complete coverage of geocoded pairs is a separate loading condition.

Flat-to-centre proximity is obtained through the flat's block. A direct flat–centre relationship would repeat the same distances for all profiles in a block. Likewise, a direct transaction–town relationship would repeat membership already available through the flat and block. These redundant relationships are omitted.

The model supports the five required analyses in Appendix A as follows.

Query	**Information and interpretation supported by the design**
Q1	Town, flat type, month, price and floor area support the mean of each sale's price per square metre for four-room and five-room flats in a selected three-year interval.
Q2	Effective remaining lease and its precision label support lease buckets, median price, mean price per square metre and transaction volume by town.
Q3	Three-room records supply floor area, town, sale year and price. Average area is transaction-weighted; the data cannot identify unique physical dwellings. The selected towns can then be compared by annual maximum sale price.
Q4	One nearest eligible-centre class per block supports NEAR and FAR price comparisons within town and flat type, without counting a sale once for every centre.
Q5	The nearest eligible centre with positive market-produce stalls supplies one market-proximity class per sale, supporting town/class averages and a budget filter.

For Q1, averaging individual price-to-area ratios differs from dividing total price by total area; the former is used. For Q4, a missing NEAR or FAR group remains unavailable rather than being assigned a zero average. For both spatial queries, sales without a known class are excluded from class-specific averages and counted separately for coverage reporting. Actual query results and performance measurements are reported in Tasks 3 and 4.

Complete conceptual ER model

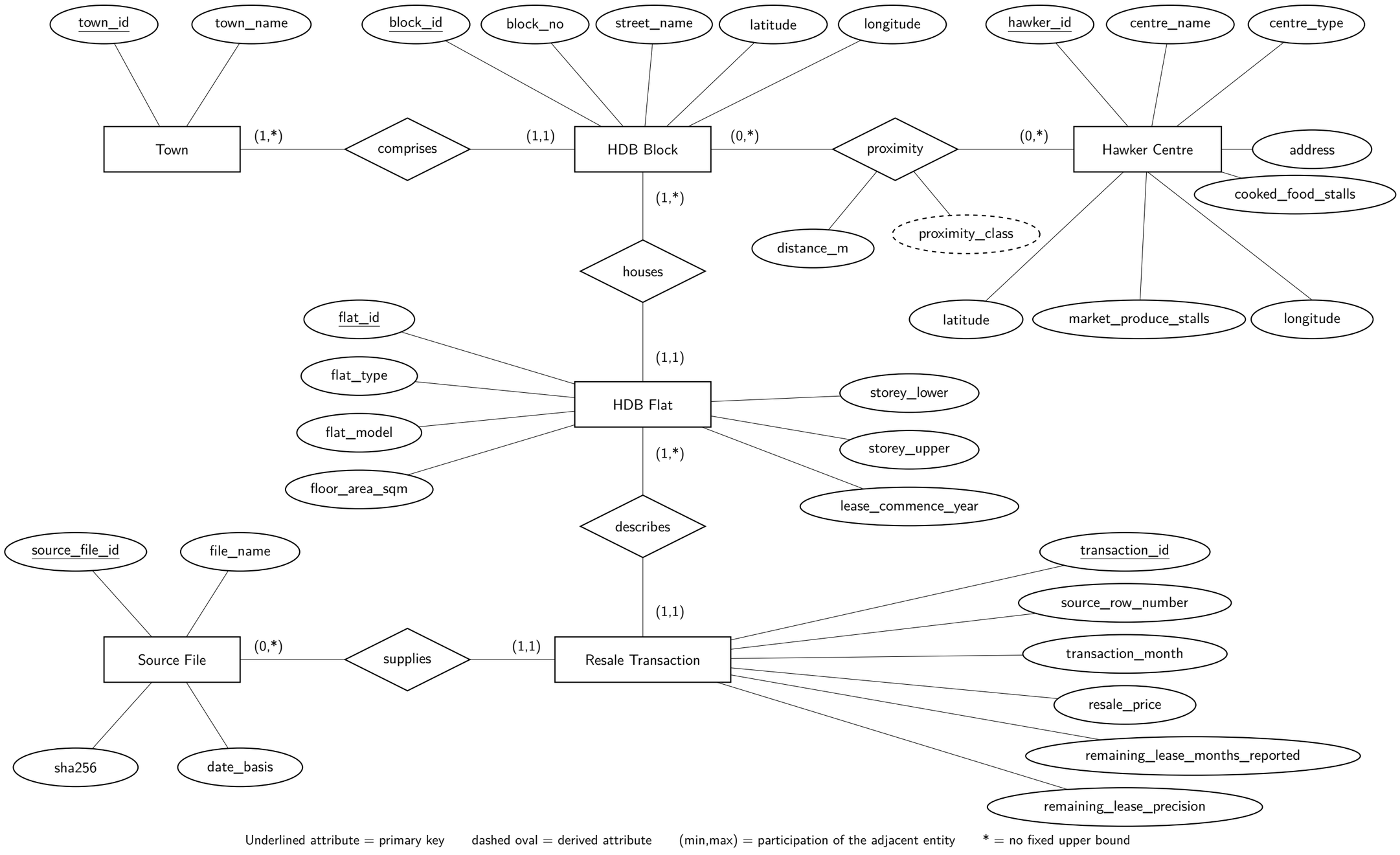


Figure 1: Complete conceptual ER model in the SD6103 lecture notation.
`proximity_class` is a derived attribute (dashed oval): it is determined by `distance_m` (NEAR below 500 m, MEDIUM from 500 m to below 1000 m, FAR from 1000 m onward) and is therefore computed in a view rather than stored. Generated identifiers are underlined as primary keys because several natural keys are composite (the flat-profile key has seven attributes); every natural key is kept as an alternate candidate key (Section 1.2.1). Source File is modelled because Q6 uses the reporting basis and because the source position distinguishes otherwise identical sale records.

1.2. Logical Schema Design and Implementation

1.2.1. Relational mapping and candidate keys

Each of the six entity types becomes a relation containing its attributes and primary key. The four one-to-many relationships are absorbed into their child relations: a block references its town, a flat profile references its block, and a transaction references its profile and source file. These foreign keys are mandatory because the corresponding child participation is $(1, 1)$. They are not unique in the child, allowing multiple children to reference the same parent. This follows the lecture mapping algorithm. [3]

The many-to-many Proximity relationship remains a separate relation, named Block Hawker Distance. Its primary key is the pair of participant identifiers, both of which are also mandatory foreign keys. The stored relation retains distance; a view supplies its computed proximity class. The final design therefore has seven stored relations.

Relation	**Primary and alternate candidate keys**
Town	Primary: town identifier. Alternate: canonical town name.
HDB Block	Primary: block identifier. Alternate: block number and street name.
HDB Flat	Primary: flat-profile identifier. Alternate: block, flat type, model, floor area, lower and upper storey bounds, and lease commencement year.
Source File	Primary: source-file identifier. Alternate: SHA256 content hash.
Resale Transaction	Primary: transaction identifier. Alternate: source-file identifier and data-record number.
Hawker Centre	Primary: centre identifier. Alternate: canonical centre name within the chosen catalogue.
Block Hawker Distance	Composite primary key: block identifier and centre identifier.

Applying the mapping gives the relational schema below. Underlined attributes form the primary key, an arrow marks a foreign key and the relation it references, and a circle marks the only nullable attributes; every other attribute is `NOT NULL`. Alternate keys are those listed in the table above.

`town(`<u>`town_id`</u>`, town_name)`

`hdb_block(`<u>`block_id`</u>`, town_id`→`town, block_no, street_name, latitude`°`, longitude`°`)`

`hdb_flat(`<u>`flat_id`</u>`, block_id`→`hdb_block, flat_type, flat_model, floor_area_sqm, storey_lower, storey_upper, lease_commence_year)`

`source_file(`<u>`source_file_id`</u>`, file_name, sha256, date_basis)`

`resale_transaction(`<u>`transaction_id`</u>`, flat_id`→`hdb_flat, source_file_id`→`source_file, source_row_number, transaction_month, resale_price, remaining_lease_months_reported`°`, remaining_lease_precision`°`)`

`hawker_centre(`<u>`hawker_id`</u>`, centre_name, address, centre_type, cooked_food_stalls, market_produce_stalls, latitude`°`, longitude`°`)`

`block_hawker_distance(`<u>`block_id`</u>`→`<u>`hdb_block`</u>`, `<u>`hawker_id`</u>`→`<u>`hawker_centre`</u>`, distance_m)`

Table 1: Relational schema (schema `housing`). Underline: primary key; →: foreign key; °: nullable.

1.2.2. Attribute domains and integrity constraints

The diagram, relational schema and SQL implementation use the same attribute names. The following definitions specify the stored fields, domains and constraints. All fields are mandatory except the paired coordinate fields and paired reported-lease fields. Entity identifiers are generated 64-bit integers, with explicit identifiers allowed for reproducible loading.

Town stores `town_id` and `town_name`. Names are unique, nonempty, trimmed and uppercase. Town names are held once, rather than repeated in transaction records.

HDB Block stores `block_id`, the foreign key `town_id`, `block_no`, `street_name`, `latitude` and `longitude`. Block numbers and street names are nonempty canonical text; their combination is unique. Coordinates are double-precision numbers, with latitude in $[-90, 90]$ and longitude in $[-180, 180]$. They are either both present or both missing. Missing geocodes do not prevent a valid transaction from participating in nonspatial analyses.

HDB Flat stores `flat_id`, the foreign key `block_id`, `flat_type`, `flat_model`, `floor_area_sqm`, `storey_lower`, `storey_upper` and `lease_commence_year`. The `storey_lower` and `storey_upper` bounds represent the published storey band, not an exact floor.

Flat type belongs to the canonical vocabulary of 1 ROOM, 2 ROOM, 3 ROOM, 4 ROOM, 5 ROOM, EXECUTIVE and MULTI-GENERATION. Model is nonempty canonical text without an unnecessarily restrictive list. Floor area is a positive finite decimal with two decimal places. Storey bounds are positive small integers, with the upper bound at least the lower bound. Lease commencement year is a four-digit year from 1900 to 9999. A block is not assumed to determine one commencement year for all its profiles.

The full profile combination listed in the key table is unique. It represents a distinct observable profile, while the generated identifier provides a compact reference from each transaction.

Source File stores `source_file_id`, `file_name`, `sha256` and `date_basis`. The file name is nonempty trimmed text. The content hash is a unique 64-character lowercase hexadecimal value, preventing identical input bytes from being registered twice under different names. The reporting basis is either APPROVAL or REGISTRATION; it records the distinction between source periods in the HDB collection. [5]

Resale Transaction stores `transaction_id`, the foreign keys `flat_id` and `source_file_id`, `source_row_number`, `transaction_month`, `resale_price`, `remaining_lease_months_reported` and `remaining_lease_precision`.

The source record number is a positive integer starting at one, excluding the header. Month is a valid date represented by its first day; it does not assert an exact sale day. Resale price is positive finite decimal currency in SGD with two decimal places. The unique source-file and record-number pair preserves distinct published records without assuming that identical characteristics indicate one sale.

Reported remaining lease is either missing or an integer number of months between zero and 1188 under the project's 99-year original-lease assumption. Precision is MONTH or YEAR and is missing exactly when the value is missing. Whole-year values must be multiples of 12 months. For example, 61 years and 4 months is 736 months with MONTH precision, while a source value of 61 years is 732 months with YEAR precision.

For older records with no reported lease, the analysis view estimates

$$L_{\text{estimate}} = 12\big[99 - (Y_{\text{sale}} - Y_{\text{commencement}})\big].$$

This is a year-resolution estimate because the commencement month is unknown. A reported value takes precedence. Estimates outside the allowed range remain unavailable rather than being silently clipped. The view records whether the effective value is reported or estimated, so Q2 can disclose the difference in precision.

Hawker Centre stores `hawker_id`, `centre_name`, `address`, `centre_type`, `cooked_food_stalls`, `market_produce_stalls`, `latitude` and `longitude`. Centre name is unique canonical text in the selected catalogue, address is nonempty trimmed text, and type is HC, MHC or MK. Stall counts are nonnegative integers. Coordinate bounds and paired availability follow the same rules as for blocks.

Total stalls and a fresh-market Boolean are not separately stored because they are determined by the retained counts. No town foreign key is added to a centre: spatial access may cross town boundaries.

Block Hawker Distance stores `block_id`, `hawker_id` and `distance_m`. Each pair occurs once. Distance is a finite, nonnegative double-precision value, bounded at 20,040,000 metres, slightly above the maximum great-circle distance for the chosen radius. The analysis view exposes its three-level class.

Why these ranges are broad. The database checks express general domain validity; the current sample's extrema are not treated as permanent business limits. Before loading, `code/prepare_data.py` accepts only finite coordinate pairs within $1.0 \leq$ latitude ≤ 1.6 and $103.5 \leq$ longitude ≤ 104.2 for both blocks and centres; invalid pairs remain NULL. This is a Singapore-area bounding box, not an exact national boundary. The year interval 1900–9999 does not establish historical plausibility, and the Earth-scale distance cap is not a local travel threshold. These generic limits avoid undocumented year or distance cutoffs. Direct SQL writes can bypass the loader's regional screening, and contextually implausible lease years still require review.

Deleting a referenced parent is restricted, so removing a town, profile or source file cannot silently remove transaction history. Constraints enforce mandatory values, keys, references, valid categories, numeric ranges and paired missingness. Source reconciliation, completeness of geocoded pairs and consistency of stored distances with current endpoint coordinates require loading validation. Changed coordinates require affected distances to be regenerated. [8]

Table 2 summarizes the declared constraints by the invalid state each one prevents.

Constraint	Where it is declared	Invalid state prevented
Data types	`numeric(12,2)` price, `numeric(8,2)` area, `smallint` storeys and lease, `date` month, `double precision` coordinates	Rounding drift in money; non-numeric values
Primary keys (7)	Generated `bigint` identifiers; composite block–centre pair	Missing or duplicate identity
Alternate keys (6)	Town name; block number and street; seven-attribute flat profile; SHA256; source file and record number; centre name	One real-world entity stored twice
Foreign keys (6)	Block to town, flat to block, sale to flat and to source file, distance to block and to centre; all `NOT NULL` with `ON DELETE/UPDATE RESTRICT`	Orphan rows; silent loss of sales history
Vocabularies	Flat type (seven values), centre type (HC, MHC, MK), date basis, lease precision	Misspelt categories such as `MULTI GENERATION`
Canonical text	Names, streets and models are non-blank and equal to `upper(btrim(...))`	Case or whitespace variants splitting one key
Ranges	Price and area positive and finite; $1 \leq$ lower storey $\leq$ upper storey; reported lease 0–1188 months, YEAR values divisible by 12; distance 0–20,040,000 m; valid latitude and longitude; month stored as its first day	Impossible values
Paired missingness	Latitude with longitude; reported lease with its precision	Half-known values

Table 2: Declared constraints: 7 primary keys, 6 alternate keys, 6 foreign keys and 30 named `CHECK` constraints. Section 1.2.4 reports the rejection tests.

1.2.3. Normalization to BCNF

BCNF is assessed on the seven stored relations obtained from our ER model. For a functional dependency $X \to Y$, rows agreeing on X must agree on Y. A dependency is nontrivial when Y contains an attribute outside X. BCNF requires the determinant of every such dependency to be a superkey. A candidate key is a minimal superkey. Having a generated primary key alone is therefore insufficient: the remaining business dependencies must also be checked. [4]

The dependency sets below describe our actual stored columns and the identity rules in Sections 1.1.1 and 1.2.1. Braces denote an attribute set. Each alternate-key dependency is shown pointing to the primary key; together with the primary-key dependency, it determines the entire row. Keys contain no NULL values. Optional coordinates and reported-lease fields are dependent values, not identifiers. The paired-missingness and numeric-range checks are additional integrity rules, not additional candidate keys.

Verification method. The listed dependencies form each relation's assumed basis. If an attribute set X contains none of its candidate keys, no dependency in the basis can add an attribute to X, so its closure is $X^+ = X$. Such a set cannot determine an attribute outside itself. Therefore every nontrivial dependency implied by the basis must have a superkey determinant. The table-by-table checks below establish that each basis has this form; they rely on business rules rather than accidental uniqueness in the current CSV rows.

Town. The relation contains `town_id` and `town_name`. Its functional dependencies are

$$\texttt{town_id} \to \texttt{town_name}; \qquad \texttt{town_name} \to \texttt{town_id}.$$

The primary key is `town_id`; the alternate candidate key is the unique, mandatory canonical `town_name`. Each determinant is a candidate key. With only these two attributes, there is no dependency involving another stored non-key attribute. Town is in BCNF.

HDB Block. Our snapshot identifies a block by its identifier or its canonical block-number/street-name pair. The dependency basis is

$$\texttt{block_id} \to \{\texttt{town_id}, \texttt{block_no}, \texttt{street_name}, \texttt{latitude}, \texttt{longitude}\};$$

$$\{\texttt{block_no}, \texttt{street_name}\} \to \texttt{block_id}.$$

Both determinants are candidate keys. A town contains many blocks; a street can contain many block numbers, and a block number can recur on different streets. None of `town_id`, `block_no` or `street_name` alone is a block key. Coordinates are not assumed unique. Town name is stored in Town, so the dependency from town identifier to town name is not repeated inside HDB Block. Under these rules, HDB Block is in BCNF.

HDB Flat. This table represents an observable attribute profile, not a uniquely identified physical dwelling. Define its complete natural profile key as

$$P = \{\texttt{block_id}, \texttt{flat_type}, \texttt{flat_model}, \texttt{floor_area_sqm}, \texttt{storey_lower}, \texttt{storey_upper}, \texttt{lease_commence_year}\}.$$

The dependency basis is $\texttt{flat_id} \to P$ and $P \to \texttt{flat_id}$. Thus the candidate keys are `flat_id` and the complete set P, enforced by the profile uniqueness constraint. No proper subset of P is declared to identify every profile. In particular, a block can contain several profiles, flat type does not fix area or model, and no uniform band width is assumed for all storey ranges.

One candidate dependency needs attention: $\texttt{block_id} \to \texttt{lease_commence_year}$. If it held, it would be a partial dependency on P and the year would belong in HDB Block. The loaded data refute it: 10,006 of the 10,009 block addresses have one commencement year, but three have two. At 21 Teban Gardens Road, 1978-lease profiles sold from 1990 to 1999 and 2013-lease profiles from 2016, consistent with a redeveloped block reusing the address; 37 Teban Gardens Road (1966/1981) and 114 Jurong East Street 13 (1981/1982) also carry two years. Since the address does not determine the year, keeping the year with the profile is required to store these sales correctly. Under the remaining identity rules, the listed determinants are keys and HDB Flat is in BCNF. Block address, coordinates and town membership are stored in HDB Block rather than repeated for each profile.

Source File. This relation identifies an imported snapshot by its generated identifier or its unique SHA256 content hash. Its dependency basis is

$$\texttt{source_file_id} \to \{\texttt{file_name}, \texttt{sha256}, \texttt{date_basis}\};$$

$$\texttt{sha256} \to \texttt{source_file_id}.$$

Both determinants are candidate keys. File name is a descriptive label and is not declared unique; different snapshots can reuse a label. Date basis is recorded for the identified snapshot, not derived by a database rule from the label. APPROVAL and REGISTRATION each apply to multiple files and do not identify a snapshot. No further functional dependency is assumed among these metadata fields, so Source File is in BCNF.

Resale Transaction. Each published record has its own identifier and a unique position within an immutable source file. The dependency basis is

$$\texttt{transaction_id} \to \{\texttt{flat_id}, \texttt{source_file_id}, \texttt{source_row_number}, \texttt{transaction_month}, \texttt{resale_price}, \texttt{remaining_lease_months_reported}, \texttt{remaining_lease_precision}\};$$

$$\{\texttt{source_file_id}, \texttt{source_row_number}\} \to \texttt{transaction_id}.$$

The determinants are the primary key and alternate candidate key respectively. A flat profile may appear in many sales, a source file contains many rows, and a row number can recur across files. Consequently, none of these attributes alone determines a transaction. Month, price and reported lease are transaction facts; they do not identify a sale. Lease precision is retained as record-level provenance, with no universal dependency from a source identifier to precision imposed by this schema. Reported values that are whole multiples of 12 months can have either YEAR or MONTH precision.

Flat properties remain in HDB Flat, and source hash, name and date basis remain in Source File. This removes dependencies from the non-key foreign keys to duplicated profile or source attributes within the transaction table. All dependencies in its declared basis have candidate-key determinants, so Resale Transaction is in BCNF.

Hawker Centre. The selected catalogue uses one canonical name per centre. The dependency basis is

$$\texttt{hawker_id} \rightarrow \{\texttt{centre_name}, \texttt{address}, \texttt{centre_type}, \texttt{cooked_food_stalls}, \texttt{market_produce_stalls}, \texttt{latitude}, \texttt{longitude}\};$$

$$\texttt{centre_name} \rightarrow \texttt{hawker_id}.$$

The identifier and canonical name are candidate keys in this snapshot. Address and coordinates are centre attributes and are not declared unique identifiers. Centre type does not fix the exact stall counts; the schema does not infer a unique centre type from the counts either. Total stalls and a fresh-market flag are omitted because they would duplicate calculations from stored counts. Under the catalogue identity rules and the listed dependency basis, Hawker Centre is in BCNF.

Block Hawker Distance. Its dependency basis is

$$\{\texttt{block_id}, \texttt{hawker_id}\} \rightarrow \texttt{distance_m}.$$

The composite determinant is the primary and only declared candidate key. A block has distances to many centres, and a centre has distances to many blocks; neither endpoint alone determines the distance. Equal numerical distances can occur for different pairs, so distance is not an identifier. The table stores no endpoint address, coordinate or other attribute that would depend on only one part of the key. Its only non-key attribute depends on the complete pair, so the relation is in BCNF. The loader separately checks the agreement of each stored distance with its endpoint coordinates.

Why the conceptual proximity class is not stored here. Our ER diagram includes `distance_m` and `proximity_class`, but the threshold rule creates the dependency

$$\texttt{distance_m} \rightarrow \texttt{proximity_class}.$$

Distance is not a superkey, and class is not part of a candidate key. Adding class to the stored distance relation would therefore violate BCNF and 3NF. Our actual implementation stores distance and computes class through `housing.proximity_class` and `v_block_hawker_proximity`. This follows the lecture treatment of computed attributes. [4] Nearest-centre classifications and other analytical calculations are exposed through views; they are not additional stored relations covered by the seven-table BCNF claim.

Conclusion for the complete design. Every determinant in the seven dependency bases is a candidate key. The closure argument above covers their implied dependencies as well. All seven stored relations are therefore in BCNF under the declared assumptions.

The mandatory foreign keys make each child-to-parent join match exactly one parent, allowing the separated facts to be reconstructed without multiplying a child row. This property is distinct from

BCNF. Any newly adopted business dependency would require the affected relation to be checked again.

1.2.4. PostgreSQL implementation and validation

Appendix A lists every command that creates the database, its seven relations, constraints, proximity-class function and analysis views; the same text is the executable `sql/01_Create_Database.sql`. The schema was populated in native PostgreSQL 18.4. Sixteen behavioural assertions and 24 expected constraint rejections passed in a separate empty database; synthetic fixtures were rolled back. The checks and evidence are in `sql/05_Validate_Schema.sql` and `results/schema_validation.json`. Required key indexes are retained; optional performance indexes are evaluated in Task 4.

2. Data Acquisition and Population

2.1. Data Acquisition

2.1.1. Source snapshots and incremental loading order

The experiment uses all five HDB resale files, the NEA hawker catalogue, the specified `hdb.csv` geolocation file and the supplied `hawker_geo.csv`. The files were frozen on 17 September 2026 (Singapore time). The source manifest records their URLs, retrieval times, column names, record counts and SHA256 hashes. The resulting population contains 986,771 published resale records, covering January 1990 to September 2026. September and the 2026 annual results are incomplete snapshots, not completed reporting periods.[5, 6, 7]

The initial instance uses the 1990–1999 file. It contains a complete three-year interval for Q1 and is smaller than the 2000–February 2012 file. One remaining file is added at each subsequent stage, in the order below. The order is deliberately not chronological; it gives five growing database instances while retaining the same Q1 date window.

Stage	New HDB period	Date basis	Added sales	Total sales
1	Jan 1990–Dec 1999	Approval	287,196	287,196
2	Mar 2012–Dec 2014	Registration	52,203	339,399
3	Jan 2015–Dec 2016	Registration	37,153	376,552
4	Jan 2017–Sep 2026	Registration	240,568	617,120
5	Jan 2000–Feb 2012	Approval	369,651	986,771

The executable downloader is `code/download_data.py`. It uses the official data.gov.sg metadata and download endpoints, retrieves the specified HDB geolocation file, and copies the supplied hawker geolocation file. Existing frozen files can be reused. The preparation program verifies their hashes before processing; changing a raw file without updating its manifest fails validation.[9]

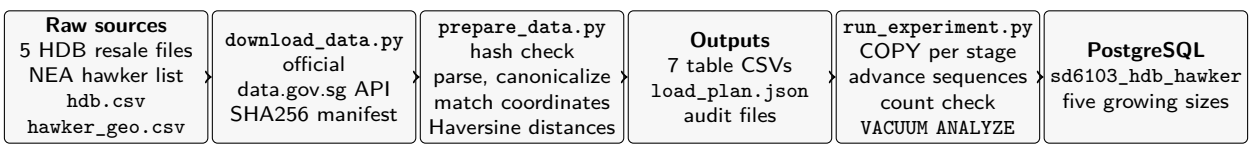


Figure 2: Automated acquisition and loading pipeline. Each stage reruns the same code on one more resale file.

2.1.2. Parsing and normalization

`code/prepare_data.py` reads CSV records with a CSV parser, preserving quoted commas in addresses. It trims text, collapses repeated whitespace and standardizes the case of town, block, street, model and flat-type identifiers. The multi-generation flat label is mapped to the schema's canonical vocabulary. Block numbers remain text, preserving letter suffixes. Standard address abbreviations, such as RD/ROAD and AVE/AVENUE, are expanded consistently for geolocation matching.

Month strings become first-of-month dates. Prices and floor areas are parsed as finite positive decimal numbers; storey bands are split into ordered lower and upper integer bounds. Source-file identity and the one-based data-record number are retained for every transaction. Generated identifiers are assigned consistently as new towns, blocks and profiles first appear. Repeated profiles share one profile row, while repeated-looking sale records remain separate transactions.

Reported lease values require three cases. The 2015–2016 file supplies whole years: multiplying by 12 preserves the value with YEAR precision. The 2017-onward file supplies years and months: these become total months with MONTH precision. Older files have no reported remaining lease and retain NULL in both reported fields. The analysis view then applies the year-resolution estimate in Section 1.2.2. No missing reported lease is silently represented as zero.

Table 3 lists, for every loaded attribute, its source column and the transformation applied.

Source column	Target attribute	Transformation
HDB resale files (all five)		
month	resale_transaction.transaction_month	YYYY-MM to first-of-month date
town	town.town_name, linked through hdb_block.town_id	Canonical text; one town per block
block, street_name	hdb_block.block_no, street_name	Canonical text; block number kept as text (letter suffixes)
flat_type	hdb_flat.flat_type	Canonical; MULTI GENERATION to MULTI-GENERATION
flat_model	hdb_flat.flat_model	Upper-cased canonical text
storey_range	hdb_flat.storey_lower, storey_upper	10 TO 12 to 10 and 12
floor_area_sqm, resale_price	hdb_flat.floor_area_sqm, resale_transaction.resale_price	Positive finite decimal, at most two places
lease_commence_date	hdb_flat.lease_commence_year	Integer year
remaining_lease (2015 onwards)	remaining_lease_months_reported, remaining_lease_precision	70 to 840 months, YEAR; 61 years 04 months to 736, MONTH; absent to NULL
File bytes and name	source_file (file_name, sha256, date_basis)	SHA256 of the frozen file; APPROVAL up to Feb 2012, REGISTRATION from Mar 2012
Record position	resale_transaction.source_row_number	One-based data-record number
Geolocation and hawker files		
hdb.csv: blk_no, street, lat, lng, addr	hdb_block.latitude, longitude	Exact block-and-street match; geocoded address must start with the expected address; Singapore bounds
NEA list: name_of_centre, location_of_centre, type_of_centre	hawker_centre.centre_name, address, centre_type	Canonical name; whitespace-collapsed address
NEA list: no_of_cooked_food_stalls, no_of_mkt_produce_stalls	cooked_food_stalls, market_produce_stalls	Integers, checked to sum to no_of_stalls; the total and owner are not stored
hawker_geo.csv: NAME, ADDRESS_MYENV, latitude, longitude	hawker_centre.latitude, longitude	Unique normalized-name match, otherwise unique postal-code match
Computed	block_hawker_distance.distance_m	Haversine for every geocoded block and centre, unrounded

Table 3: Source-to-target mapping of every loaded attribute.

2.1.3. Geolocation reconciliation and distance computation

For HDB blocks, matching uses the canonical block number and street name. The program additionally checks that the geolocation file's resolved address starts with the corresponding expanded block-and-street address. This catches records whose original address and returned geocode refer to different locations. It also checks coordinate validity and rejects conflicting matches. Of 12,442 geolocation source rows, 29 are excluded by these checks. No approximate text match is used to assign a doubtful coordinate.

For hawkers, the 107-row NEA catalogue defines the database population. A unique normalized

name match supplies 19 coordinates, and a unique postal-code match supplies another 85. Three catalogue entries remain unmatched: Market Street Food Centre, Blk 4A Woodlands Centre Road, and Blks 1A/2A/3A Commonwealth Drive. A relocated centre or an ambiguous multi-block address is not assigned a coordinate merely because its name looks similar. The 129-row geolocation file enriches the catalogue; it does not create centres with unknown or invented stall counts.

Coverage measure	**Matched**	**Total**
HDB blocks	9,817	10,009
Catalogue centres	104	107
Transactions with a matched block	971,501	986,771

Thus 98.45% of transactions can be classified spatially. The remaining 15,270 transactions are retained for Q1–Q3 but excluded from the proximity analyses; missing coordinates never imply FAR. Missing centre coordinates can also make the nearest observed centre farther away than the true nearest centre. The coverage files identify every unresolved block and centre, making these limitations inspectable.

The program computes the Haversine distance from each matched block to every matched catalogue centre. It materializes $9{,}817 \times 104 = 1{,}020{,}968$ block–centre pairs, including pairs beyond one kilometre and market-only centres. The query views apply centre eligibility separately: 101 geocoded HC/MHC centres qualify for Q4, and 78 of these have fresh-market stalls for Q5. Distances are not rounded before classification or nearest-centre selection.

This complete pair relation prevents the FAR category from disappearing through a distance cut-off. At each stage, only the distance rows for newly encountered blocks are appended. Centre information is loaded once. The distances are calculated once per block–centre pair, rather than once per sale.

The NEA catalogue is historical, while the available geolocation files represent their own snapshots. The database therefore supports a comparison with the supplied location information, not reconstruction of hawker opening, closure or relocation dates. Historical nominal prices are pooled unless a query explicitly restricts the period. Differences cannot be interpreted as inflation-adjusted or causal effects of access to food facilities.

2.2. Database Population

2.2.1. Bulk loading and final relation sizes

The preparation program produces one CSV per stored relation and a `load_plan.json` file containing each stage's new and cumulative row counts. The experiment runner bulk-loads the appropriate new CSV records through PostgreSQL COPY. Parents are loaded before their dependent records: Town, HDB Block, HDB Flat and Source File precede Resale Transaction; Hawker Centre precedes the distance relation. Each stage is committed as one transaction, so a failed stage cannot leave a partly loaded increment.[10]

After loading explicit identifiers, the identity sequences are advanced to the loaded maxima. Actual table counts are compared with the load plan, and VACUUM ANALYZE updates planner statistics and visibility information. All declared primary keys, alternate keys, foreign keys and checks remain enabled. The optional `sql/02_Load_Data.sql` performs a single full load for users who want the final database without rerunning the five-stage experiment.

Stored relation	Final rows
Town	27
HDB Block	10,009
HDB Flat (observable profiles)	146,851
Source File	5
Resale Transaction	986,771
Hawker Centre	107
Block Hawker Distance	1,020,968

2.2.2. Issues encountered and their resolution

Acquisition problem encountered. During data acquisition, direct Python HTTP requests to data.gov.sg were refused. The downloader was changed to call the official metadata and download endpoints through curl with retries and a timeout. This problem and its workaround are documented in `code/download_data.py`; the submitted files do not retain the original HTTP status or exception text.

Preparation and loading safeguards. Before COPY, the loader maps `MULTI GENERATION` to `MULTI-GENERATION`; the original spelling would violate `flat_type_valid`. After each COPY stage, it advances identity sequences to the loaded maxima, preventing later generated identifiers from colliding with explicitly loaded keys. The supplied records do not identify an original check rejection or sequence collision at a particular stage, so these are reported as safeguards rather than reconstructed failure events. Table 4 summarizes the source-data issues found during preparation, their effects or risks, and the implemented resolutions.

Issue found during preparation	Effect or risk	Resolution
Flat type written `MULTI GENERATION` in 1990–1999 and `MULTI-GENERATION` later	The older spelling would fail `flat_type_valid` and split one category into two labels	Mapped to the canonical `MULTI-GENERATION`
Flat model case changes: `IMPROVED` before 2000, `Improved` afterwards	Mixed case would fail `flat_model_canonical` or split one profile into two spellings	Trimmed and upper-cased before the profile lookup
Remaining lease absent before 2015, whole years (`70`) in 2015–2016, `61 years 04 months` from 2017	Text is not a valid `smallint`; `70` would be read as 70 months; a missing value stored as 0 would enter the shortest bucket	Parsed to total months with YEAR or MONTH precision; missing values stay NULL and are estimated only in the view (277,721 reported, 708,999 estimated, 51 UNKNOWN)
Month as `YYYY-MM`; storey band as `10 TO 12`	Neither is a complete date or a number	First-of-month date; band split into ordered lower and upper bounds
Three block addresses with two lease commencement years	Would conflict if the year were a block attribute	Year kept on the flat profile (Section 1.2.3)
1,929 records identical in every source field	A key on observable attributes would reject them or silently merge distinct published sales	Retained as separate transactions keyed by source file and record number
Street abbreviations; 29 `hdb.csv` rows geocoded to another address (e.g. 1 Everton Park resolved to 1 Cantonment Road)	Blocks without coordinates or with wrong ones	Abbreviations expanded; the geocoded address must start with the expected address; rejected rows logged; 192 blocks (1.55% of sales) keep NULL coordinates
Hawker names differ between the NEA list and `hawker_geo.csv`	A name join leaves most centres without coordinates	Unique normalized-name match (19), then unique postal-code match (85); three centres keep NULL coordinates
Quoted commas in hawker addresses	A naive split shifts columns	Records read with a CSV parser

Table 4: Source-data issues identified during preparation, their effects or risks, and their resolutions.

Outcome of the final run. After the acquisition workaround, data preparation and loading safeguards were in place, all five COPY stages completed without a constraint or parsing rejection, and table counts matched the load plan after every stage. Separately, synthetic records produced 24 expected constraint rejections during schema validation (Section 1.2.4); these were deliberate tests, not failures of the final data load. The rejected geolocation rows, unmatched blocks and centres, and the town-conflict check (none found) are logged in `results/data_quality/`. Repeated profile attributes are consolidated into HDB Flat, while repeated-looking sale records remain separate because the data cannot establish that they duplicate one physical sale.

Reproducibility. A second preparation run into a separate directory produced byte-identical copies of all seven table CSVs. The hashes are recorded in `results/reproducibility_validation.json`. This checks deterministic transformation of the frozen inputs; a fresh download of a subsequently revised source is not expected to have identical bytes.

3. SQL Queries and Initial Performance Results

3.1. SQL Queries

The six statements below are reproduced exactly as executed from `sql/03_Queries.sql`; each was run at all five database sizes. The discussion below uses the full database, with complete outputs in `results/queries/stage_5/`. The pgAdmin Query Tool screenshots show each query running against the full database, together with the first displayed result rows, column labels and total row count.

All six results passed an independent pandas recomputation from the prepared CSVs: keys, missing values, counts and aggregates agreed within rounding precision. SQL checks also verified coverage, nearest-centre uniqueness and the query-specific output conditions. Evidence is in `results/independent_validation.json`.

3.1.1. Q1: Price per square metre across towns

Q1 selects 4 ROOM and 5 ROOM transactions from 1 January 1997 up to, but not including, 1 January 2000: exactly 36 months within the initial file. It groups by town and flat type and averages each sale's price divided by its own area. This is a transaction-weighted mean of ratios, not total price divided by total area. Counts make small groups visible. The 48 groups contain 77,877 sales, unchanged throughout the growth experiment.

Q1 SQL

```sql
-- Average of individual price/area ratios, not total price / total area.
SELECT t.town_name, f.flat_type,
       round(avg(r.resale_price / f.floor_area_sqm), 2) AS avg_price_per_sqm,
       count(*) AS transaction_volume
FROM housing.resale_transaction AS r
JOIN housing.hdb_flat AS f USING (flat_id)
JOIN housing.hdb_block AS b USING (block_id)
JOIN housing.town AS t USING (town_id)
WHERE r.transaction_month >= DATE '1997-01-01'
  AND r.transaction_month < DATE '2000-01-01'
  AND f.flat_type IN ('4 ROOM', '5 ROOM')
GROUP BY t.town_name, f.flat_type
ORDER BY t.town_name, f.flat_type;
```

Query Query History

```sql
-- Average of individual price/area ratios, not total price / total area.
SELECT t.town_name, f.flat_type,
       round(avg(r.resale_price / f.floor_area_sqm), 2) AS avg_price_per_sqm,
       count(*) AS transaction_volume

FROM housing.resale_transaction AS r
JOIN housing.hdb_flat AS f USING (flat_id)
JOIN housing.hdb_block AS b USING (block_id)
JOIN housing.town AS t USING (town_id)

WHERE r.transaction_month >= DATE '1997-01-01'
  AND r.transaction_month < DATE '2000-01-01'
  AND f.flat_type IN ('4 ROOM', '5 ROOM')
```

Data Output Messages Notifications

Showing rows: 1 to 48 Page No: 1 of 1

	town_name text	flat_type text	avg_price_per_sqm numeric	transaction_volume bigint
1	ANG MO KIO	4 ROOM	2882.18	2278
2	ANG MO KIO	5 ROOM	3388.97	771
3	BEDOK	4 ROOM	2755.77	3845
4	BEDOK	5 ROOM	3121.32	1919
5	BISHAN	4 ROOM	3376.82	2133
6	BISHAN	5 ROOM	3748.41	1119
7	BUKIT BATOK	4 ROOM	2474.27	3134
8	BUKIT BATOK	5 ROOM	2877.30	692
9	BUKIT MERAH	4 ROOM	3246.71	988
10	BUKIT MERAH	5 ROOM	3689.07	735
11	BUKIT PANJANG	4 ROOM	2250.12	1478
12	BUKIT PANJANG	5 ROOM	2623.83	630

Total rows: 48 Query complete 00:00:00.121 CRLF Ln 1, Col 74

Figure 3: Q1 result excerpt: the first displayed rows of 48 town–flat-type groups.

Insight. Marine Parade has the highest mean price per square metre among the 48 groups: SGD 4,021.43 for five-room flats (294 sales), ahead of Queenstown (SGD 3,856.82) and Bishan (SGD 3,748.41) five-room flats. Five-room flats have the higher mean in 22 of the 23 towns that sold both types, with a median ratio of 1.14; Central Area is the only exception (SGD 3,451.81 for four-room against SGD 3,317.39 for five-room). The lowest means come from very small groups, Lim Chu Kang four-room (SGD 1,606.43, 3 sales) and Sengkang four-room (SGD 2,031.62, 11 sales), so transaction counts matter when comparing towns.

3.1.2. Q2: Remaining lease versus transaction price

Q2 groups each town's sales into remaining-lease intervals $[0, 50)$, $[50, 70)$, $[70, 90)$ and $[90, 99]$ years. The implementation compares total months with 600, 840 and 1080, avoiding boundary ambiguity. UNKNOWN is an additional missing-data group, not a fifth numerical interval. Each group reports its median resale price, mean individual price per square metre and transaction volume. Reported and estimated lease counts expose the quality of the lease information.

The continuous median interpolates between the two central prices when a group has an even number of transactions. All 986,771 records appear exactly once across the 102 output groups. The 51 unavailable effective leases remain visible rather than being silently dropped or assigned to the shortest bucket.

Q2 SQL

```sql
-- Lease thresholds are half-open: [0,50), [50,70), [70,90), [90,99].
```

```sql
-- Preserve an explicit UNKNOWN group if an effective lease is unavailable.
WITH bucketed AS (
    SELECT *, CASE
        WHEN remaining_lease_months IS NULL THEN 'UNKNOWN'
        WHEN remaining_lease_months < 600 THEN '0-<50 years'
        WHEN remaining_lease_months < 840 THEN '50-<70 years'
        WHEN remaining_lease_months < 1080 THEN '70-<90 years'
        ELSE '90-99 years' END AS lease_bucket
    FROM housing.v_resale_analysis
)
SELECT town_name, lease_bucket,
       round((percentile_cont(0.5) WITHIN GROUP (ORDER BY resale_price))::numeric, 2)
           AS median_resale_price,
       round(avg(price_per_sqm), 2) AS avg_price_per_sqm,
       count(*) AS transaction_volume,
       count(*) FILTER (WHERE remaining_lease_basis IN ('REPORTED_MONTH','REPORTED_YEAR'))
           AS reported_lease_count,
       count(*) FILTER (WHERE remaining_lease_basis = 'ESTIMATED_YEAR') AS estimated_lease_count
FROM bucketed
GROUP BY town_name, lease_bucket
ORDER BY town_name, lease_bucket;
```

Query | Query History

```sql
-- QUERY Q2
-- Lease thresholds are half-open: [0,50), [50,70), [70,90), [90,99].
-- Preserve an explicit UNKNOWN group if an effective lease is unavailable.
WITH bucketed AS (
    SELECT *, CASE
        WHEN remaining_lease_months IS NULL THEN 'UNKNOWN'
        WHEN remaining_lease_months < 600 THEN '0-<50 years'
        WHEN remaining_lease_months < 840 THEN '50-<70 years'
        WHEN remaining_lease_months < 1080 THEN '70-<90 years'
        ELSE '90-99 years' END AS lease_bucket
```

Data Output | Messages | Notifications

Showing rows: 1 to 102 | Page No: 1 of 1

	town_name text	lease_bucket text	median_resale_price numeric	avg_price_per_sqm numeric	transaction_volume bigint	reported_lease_count bigint	estimated_lease_count bigint
1	ANG MO KIO	0-<50 years	432000.00	5952.76	57	57	0
2	ANG MO KIO	50-<70 years	360000.00	4798.44	15096	9358	5738
3	ANG MO KIO	70-<90 years	175000.00	2456.05	36391	1174	35217
4	ANG MO KIO	90-99 years	470000.00	5048.14	2774	962	1812
5	ANG MO KIO	UNKNOWN	324000.00	3492.22	6	0	0
6	BEDOK	0-<50 years	385000.00	5748.45	193	193	0
7	BEDOK	50-<70 years	365000.00	4707.00	18040	11708	6332
8	BEDOK	70-<90 years	200000.00	2602.29	44579	2118	42461
9	BEDOK	90-99 years	335000.00	3324.52	6775	967	5808
10	BEDOK	UNKNOWN	575000.00	4166.67	1	0	0
11	BISHAN	0-<50 years	367944.00	5357.94	56	56	0
12	BISHAN	50-<70 years	660000.00	6349.18	3528	3277	251
13	BISHAN	70-<90 years	385000.00	3887.79	12236	1402	10834
14	BISHAN	90-99 years	368000.00	3269.99	6352	136	6216

Total rows: 102 | Query complete 00:00:04.073 | CRLF | Ln 23, Col 34

Figure 4: Q2 result excerpt, including the explicit UNKNOWN lease group.

Insight. In Ang Mo Kio, the median is SGD 432,000 for leases below 50 years, SGD 360,000 for 50–under 70 years, SGD 175,000 for 70–under 90 years, and SGD 470,000 for 90–99 years. Prices are therefore not monotonic across these pooled lease categories. The shortest bucket contains only 57 transactions, while the 70–under 90-year bucket contains 36,391, of which 35,217 rely on an estimated lease. The pattern is systematic rather than specific to Ang Mo Kio: the 70–under 90-year bucket has the lowest median in 19 of the 27 towns, and the 50–under 70-year bucket the highest in 12. Bucket composition explains it. 82% of the 553,902 sales in the 70–under 90-year bucket have estimated leases, so they come from the 1990–2014 files with lower nominal prices, whereas all 7,830 sales below 50 years have reported leases and therefore date from 2015 onwards.

This does not imply that shorter leases cause higher values. The buckets mix sale years, flat types and properties, and older low nominal prices can enter longer-lease categories. A causal or valuation analysis would need to control for sale period and other attributes and to examine lease uncertainty. Q2 provides the requested descriptive comparison, with the relevant data limitations made visible.

3.1.3. Q3: Annual maximum prices in towns with large flats

Q3 first ranks towns by their mean floor area among recorded three-room sale transactions. The population is transaction-weighted: a profile sold several times contributes once per recorded sale. It is not a census of physical dwellings, which cannot be identified uniquely from these sources. The top ten towns are selected using descending mean area, with town name as a deterministic tie-breaker. Their three-room sales are then grouped by town and year to obtain the annual maximum resale price and volume.

Q3 SQL

```sql
-- Transaction-weighted average area; tie-break by town name for deterministic top 10.
WITH top_towns AS (
    SELECT b.town_id, t.town_name, avg(f.floor_area_sqm) AS avg_floor_area_sqm
    FROM housing.resale_transaction AS r
    JOIN housing.hdb_flat AS f USING (flat_id)
    JOIN housing.hdb_block AS b USING (block_id)
    JOIN housing.town AS t USING (town_id)
    WHERE f.flat_type = '3 ROOM'
    GROUP BY b.town_id, t.town_name
    ORDER BY avg_floor_area_sqm DESC, t.town_name
    LIMIT 10
)
SELECT tt.town_name, round(tt.avg_floor_area_sqm, 2) AS avg_floor_area_sqm,
       EXTRACT(YEAR FROM r.transaction_month)::integer AS sale_year,
       max(r.resale_price) AS highest_resale_price,
       count(*) AS transaction_volume
FROM top_towns AS tt
JOIN housing.hdb_block AS b ON b.town_id = tt.town_id
JOIN housing.hdb_flat AS f USING (block_id)
JOIN housing.resale_transaction AS r USING (flat_id)
WHERE f.flat_type = '3 ROOM'
GROUP BY tt.town_name, tt.avg_floor_area_sqm, EXTRACT(YEAR FROM r.transaction_month)
ORDER BY tt.avg_floor_area_sqm DESC, tt.town_name, sale_year;
```

Query Query History

```sql
-- Transaction-weighted average area; tie-break by town name for deterministic top 10.
WITH top_towns AS (
    SELECT b.town_id, t.town_name, avg(f.floor_area_sqm) AS avg_floor_area_sqm
    FROM housing.resale_transaction AS r
    JOIN housing.hdb_flat AS f USING (flat_id)
    JOIN housing.hdb_block AS b USING (block_id)
    JOIN housing.town AS t USING (town_id)
    WHERE f.flat_type = '3 ROOM'
    GROUP BY b.town_id, t.town_name
    ORDER BY avg_floor_area_sqm DESC, t.town_name
```

Data Output Messages Notifications

Showing rows: 1 to 369 Page No: 1 of 1

	town_name text	avg_floor_area_sqm numeric	sale_year integer	highest_resale_price numeric	transaction_volume bigint
1	TAMPINES	72.51	1990	67000.00	310
2	TAMPINES	72.51	1991	72000.00	232
3	TAMPINES	72.51	1992	83000.00	523
4	TAMPINES	72.51	1993	121000.00	642
5	TAMPINES	72.51	1994	143000.00	839
6	TAMPINES	72.51	1995	180000.00	930
7	TAMPINES	72.51	1996	265000.00	1187
8	TAMPINES	72.51	1997	290000.00	844
9	TAMPINES	72.51	1998	243000.00	1289
10	TAMPINES	72.51	1999	245000.00	1325
11	TAMPINES	72.51	2000	265000.00	855
12	TAMPINES	72.51	2001	230000.00	877
13	TAMPINES	72.51	2002	233000.00	836
14	TAMPINES	72.51	2003	238000.00	695

Total rows: 369 Query complete 00:00:00.574 CRLF Ln 25, Col 1

Figure 5: Q3 result excerpt: early annual maxima for Tampines.

Insight. Tampines ranks first with a mean three-room area of 72.51 m^2, followed by Choa Chu Kang (71.88 m^2) and Bukit Panjang (71.76 m^2). The other selected towns are Ang Mo Kio, Jurong East, Woodlands, Clementi, Yishun, Bukit Timah and Bukit Batok. The result contains 369 town–year observations; missing town–year combinations are not filled with invented zero prices.

Figure 6 shows that the ten towns follow one market cycle. In the nine towns with 1990 sales, the annual maximum rose 3.5 to 4.7 times between 1990 and 1997. The 1997 maximum was not exceeded again until 2007–2008 in seven towns (2000 in Bukit Timah, 2004 in Bukit Panjang and Choa Chu Kang). Five towns recorded a lower maximum in 2019 than in 2012, before all ten rose sharply to 2025. Large three-room flats do not imply the highest prices: in 2025, the last complete year, Ang Mo Kio has the highest maximum (SGD 794,500) and Choa Chu Kang, second by floor area, the lowest (SGD 530,000). The overall highest value, SGD 896,000 in Clementi in 2026, comes from an incomplete year, and a maximum is an extreme observation rather than a representative price.

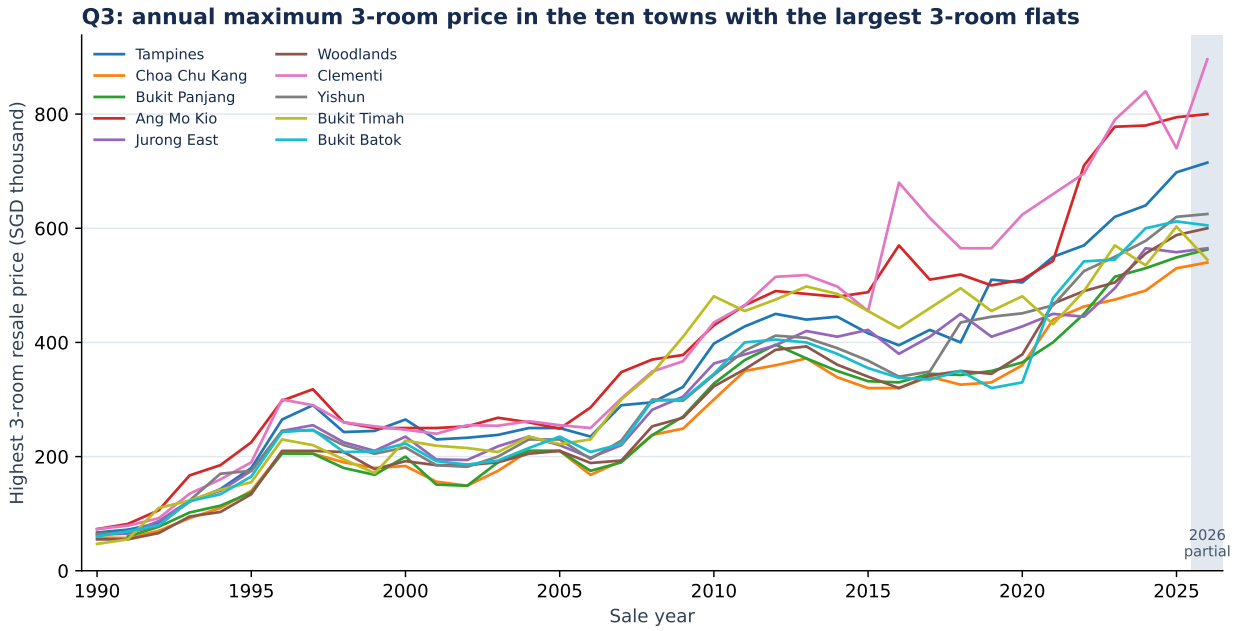


Figure 6: Q3 annual maxima for all ten selected towns, plotted from the complete Q3 result. The shaded 2026 column is a partial year.

3.1.4. Q4: Hawker-centre proximity effect

Q4 assigns each geocoded block to its nearest eligible HC/MHC centre before joining to sales. This prevents a sale from contributing once for every nearby centre. The query computes conditional means for NEAR and FAR within each town and flat type. MEDIUM sales do not enter either comparison mean. The output follows the required five-column schema, and the percentage is calculated from unrounded means:

$$\text{difference}(\%) = 100\,\frac{\overline{P}_{\text{NEAR}} - \overline{P}_{\text{FAR}}}{\overline{P}_{\text{FAR}}}.$$

If either comparison group is absent, its mean and the percentage remain NULL. This distinguishes an unavailable comparison from a genuine zero difference. Positive percentages indicate a higher mean in NEAR; negative percentages indicate a lower mean.

Q4 SQL

```
-- The view selects one nearest eligible centre per block; a sale is never multiplied.
-- An absent NEAR/FAR comparison remains NULL, not zero.
WITH averages AS (
    SELECT t.town_name AS town, f.flat_type,
           avg(r.resale_price) FILTER (WHERE n.proximity_class = 'NEAR') AS near_price,
           avg(r.resale_price) FILTER (WHERE n.proximity_class = 'FAR') AS far_price
    FROM housing.resale_transaction AS r
    JOIN housing.hdb_flat AS f USING (flat_id)
    JOIN housing.hdb_block AS b USING (block_id)
    JOIN housing.town AS t USING (town_id)
    JOIN housing.v_nearest_hawker AS n USING (block_id)
    GROUP BY t.town_name, f.flat_type
)
SELECT town, flat_type,
       round(near_price, 2) AS avg_price_near_hawker,
```

```
       round(far_price, 2) AS avg_price_far_hawker,
       round((near_price - far_price) * 100 / NULLIF(far_price, 0), 2) AS price_diff_percentage
FROM averages
ORDER BY town, flat_type;
```

Query Query History

```
1  -- The view selects one nearest eligible centre per block; a sale is never multiplied.
2  -- An absent NEAR/FAR comparison remains NULL, not zero.
3  WITH averages AS (
4      SELECT t.town_name AS town, f.flat_type,
5             avg(r.resale_price) FILTER (WHERE n.proximity_class = 'NEAR') AS near_price,
6             avg(r.resale_price) FILTER (WHERE n.proximity_class = 'FAR') AS far_price
7      FROM housing.resale_transaction AS r
8      JOIN housing.hdb_flat AS f USING (flat_id)
9      JOIN housing.hdb_block AS b USING (block_id)
10     JOIN housing.town AS t USING (town_id)
```

Data Output Messages Notifications

SQL Showing rows: 1 to 131 Page No: 1 of 1

	town text	flat_type text	avg_price_near_hawker numeric	avg_price_far_hawker numeric	price_diff_percentage numeric
1	ANG MO KIO	2 ROOM	174981.74	[null]	[null]
2	ANG MO KIO	3 ROOM	208352.16	[null]	[null]
3	ANG MO KIO	4 ROOM	337169.97	[null]	[null]
4	ANG MO KIO	5 ROOM	503444.00	[null]	[null]
5	ANG MO KIO	EXECUTIVE	684774.77	[null]	[null]
6	BEDOK	2 ROOM	150224.79	[null]	[null]
7	BEDOK	3 ROOM	203451.17	[null]	[null]
8	BEDOK	4 ROOM	300522.17	[null]	[null]
9	BEDOK	5 ROOM	411692.38	[null]	[null]
10	BEDOK	EXECUTIVE	542821.31	561661.15	-3.35
11	BISHAN	3 ROOM	219672.77	260645.50	-15.72
12	BISHAN	4 ROOM	344024.83	389467.18	-11.67
13	BISHAN	5 ROOM	481314.08	528516.49	-8.93
14	BISHAN	EXECUTIVE	639376.87	668136.32	-4.30

Total rows: 131 Query complete 00:00:12.669 CRLF Ln 1, Col 1

Figure 7: Q4 result excerpt; NULL records the absence of a NEAR or FAR comparison group.

Insight. Of 131 town–flat-type rows, 53 have both comparison groups: 11 show a positive difference and 42 a negative difference. The remaining 78 cannot support the percentage comparison. For Bedok executive flats, the NEAR mean is SGD 542,821.31 and the FAR mean is SGD 561,661.15, giving -3.35%.

The result provides no consistent positive proximity premium across the comparable groups. It also cannot establish a causal effect: dates, lease lengths, flat models and other location attributes differ between groups, prices span several decades, and historical hawker availability is not observed. Excluding unmatched blocks and centres may further affect the comparison.

Sensitivity to the hawker population. The brief designates the NEA list as the hawker-centre dataset, so the database contains its 107 centres. `hawker_geo.csv` also locates 25 centres that are not in that list, including newer centres such as Pasir Ris Central, Our Tampines Hub, One Punggol and Bukit Panjang. Without them, every geocoded sale in Bukit Panjang, Choa Chu Kang, Punggol, Sembawang and Sengkang is FAR, which explains many of the NULL comparisons. We repeated the Q4 classification with these 25 centres added as eligible, using `code/hawker_sensitivity.py` without changing the database. This moves 275,087 of the 971,501 geocoded sales to a nearer class and yields 74 comparable rows: 41 positive and 33 negative, with a median difference of $+2.1\%$ against -9.0% as submitted. Restricting both versions to sales from 2017 gives the same reversal (16 positive and 37 negative, against 44 and 30). The sign of the Q4 differential therefore depends on which centres are counted, so we read Q4 as showing no robust proximity effect in either direction.

3.1.5. Q5: Town-level prices near fresh-market hawkers

Q5 restricts candidate centres to HC/MHC records with a positive market-produce stall count, and only then finds the nearest centre for each block. It averages sale prices by town and the resulting NEAR, MEDIUM or FAR class. A fixed budget threshold of SGD 250,000 is applied to the unrounded class average. A town is returned when at least one of its class averages exceeds the threshold; only its qualifying class rows are displayed.

This interpretation applies the budget to the grouping explicitly requested in the question. It does not filter individual transactions above the budget before averaging, and it does not require every class in a town to exceed the threshold. The threshold is held constant for all five performance stages. Displaying only the qualifying class rows keeps the output consistent with the filter: every returned average exceeds the budget, and a missing class row shows where that town's average is at or below it.

Q5 SQL

```sql
-- Eligible HC/MHC centres with market stalls are filtered BEFORE nearest selection.
-- HAVING applies the budget to each town/proximity-class average.
SELECT t.town_name, n.proximity_class,
       round(avg(r.resale_price), 2) AS avg_resale_price,
       count(*) AS transaction_volume
FROM housing.resale_transaction AS r
JOIN housing.hdb_flat AS f USING (flat_id)
JOIN housing.hdb_block AS b USING (block_id)
JOIN housing.town AS t USING (town_id)
JOIN housing.v_nearest_market_hawker AS n USING (block_id)
GROUP BY t.town_name, n.proximity_class
HAVING avg(r.resale_price) > 250000
ORDER BY t.town_name,
         CASE n.proximity_class WHEN 'NEAR' THEN 1 WHEN 'MEDIUM' THEN 2 ELSE 3 END;
```

Query | Query History

```sql
-- Eligible HC/MHC centres with market stalls are filtered BEFORE nearest selection.
-- HAVING applies the budget to each town/proximity-class average.
SELECT t.town_name, n.proximity_class,
       round(avg(r.resale_price), 2) AS avg_resale_price,
       count(*) AS transaction_volume
FROM housing.resale_transaction AS r
JOIN housing.hdb_flat AS f USING (flat_id)
JOIN housing.hdb_block AS b USING (block_id)
JOIN housing.town AS t USING (town_id)
JOIN housing.v_nearest_market_hawker AS n USING (block_id)
```

Data Output | Messages | Notifications

Showing rows: 1 to 56 | Page No: 1 of 1

	town_name text	proximity_class text	avg_resale_price numeric	transaction_volume bigint
1	ANG MO KIO	NEAR	275542.04	49529
2	ANG MO KIO	MEDIUM	268269.28	3825
3	BEDOK	NEAR	269496.99	50202
4	BEDOK	MEDIUM	341963.58	18378
5	BEDOK	FAR	561661.15	515
6	BISHAN	NEAR	331907.65	2645
7	BISHAN	MEDIUM	456548.22	16480
8	BISHAN	FAR	448523.36	3048
9	BUKIT BATOK	NEAR	279147.89	1219
10	BUKIT BATOK	MEDIUM	307462.59	4631
11	BUKIT BATOK	FAR	305600.16	41314
12	BUKIT MERAH	NEAR	380949.70	23049
13	BUKIT MERAH	MEDIUM	420028.69	8007
14	BUKIT MERAH	FAR	462715.70	2994

Total rows: 56 | Query complete 00:00:06.741 | CRLF | Ln 1, Col 1

Figure 8: Q5 result excerpt: town–proximity averages above SGD 250,000.

Insight. The final query returns 56 class rows covering 26 of the 27 towns; Lim Chu Kang is excluded. The condition is also a proper subset at each earlier stage: 14/26, 21/27, 23/27 and 26/27 towns. Thus the chosen budget satisfies the assignment condition at every measured database size.

For Ang Mo Kio, NEAR and MEDIUM average SGD 275,542.04 and SGD 268,269.28 respectively, both exceeding the threshold. For Bedok, the FAR group averages SGD 561,661.15 across only 515 transactions, compared with SGD 269,496.99 across 50,202 NEAR transactions. Group composition and sample size matter; a high class average does not imply that every flat in that class costs more than the budget. Across the 26 returned towns, 12 return all three classes and 7 return only FAR, and FAR has the highest class average in 18 towns. This agrees with Q4 and its sensitivity analysis: FAR blocks are concentrated in newer towns that the NEA list does not serve, so a high FAR average is more likely to reflect where newer, more expensive flats are than an effect of distance to a market.

3.1.6. Q6: Nearest-centre catchments and reporting basis

The additional question is: *Which nearest-centre catchments account for the most recorded sales, by town and source reporting basis, and what are their average price per square metre and distance?*

Q6 ranks valid block–centre distances, selects one nearest eligible centre per block, and joins each sale to its profile, block, town, source file and selected centre. It therefore uses all seven stored tables: Town, HDB Block, HDB Flat, Resale Transaction, Source File, Hawker Centre and Block Hawker Distance. Source File has a substantive role: it separates approval-date and registration-date records. Q6 ranks the distances in its own common table expression instead of reading `v_nearest_hawker`, so that Block Hawker Distance and Hawker Centre appear explicitly in the statement; the ranking rule is identical to the view's.

The result groups by town, centre and date basis, orders by descending transaction volume and returns the largest 20 groups. Ties are resolved by town, centre name and date basis. Average distance is weighted by sales, matching the transaction-weighted price measure; it is not the unweighted average distance of all blocks in a catchment.

Q6 SQL

```sql
-- Additional question: Which nearest-centre catchments account for the most sales,
-- by town and source reporting basis, and what are their average price/m2 and distance?
-- All seven stored tables participate. Select one centre per block before aggregating.
WITH ranked_distances AS (
    SELECT d.block_id, d.hawker_id, d.distance_m,
           row_number() OVER (PARTITION BY d.block_id ORDER BY d.distance_m, d.hawker_id) AS rn
    FROM housing.block_hawker_distance AS d
    JOIN housing.hawker_centre AS eligible USING (hawker_id)
    WHERE eligible.centre_type IN ('HC', 'MHC')
)
SELECT t.town_name, h.centre_name, s.date_basis,
       count(*) AS transaction_volume,
       round(avg(r.resale_price / f.floor_area_sqm), 2) AS avg_price_per_sqm,
       round(avg(n.distance_m)::numeric, 1) AS avg_distance_m
FROM housing.resale_transaction AS r
JOIN housing.hdb_flat AS f USING (flat_id)
JOIN housing.hdb_block AS b USING (block_id)
JOIN housing.town AS t USING (town_id)
JOIN housing.source_file AS s USING (source_file_id)
JOIN ranked_distances AS n ON n.block_id = b.block_id AND n.rn = 1
JOIN housing.hawker_centre AS h ON h.hawker_id = n.hawker_id
GROUP BY t.town_name, h.centre_name, s.date_basis
ORDER BY transaction_volume DESC, t.town_name, h.centre_name, s.date_basis
```

```
LIMIT 20;
```

Query Query History

```
WITH ranked_distances AS (
    SELECT d.block_id, d.hawker_id, d.distance_m,
           row_number() OVER (PARTITION BY d.block_id ORDER BY d.distance_m, d.hawker_id) AS rn
    FROM housing.block_hawker_distance AS d
    JOIN housing.hawker_centre AS eligible USING (hawker_id)
    WHERE eligible.centre_type IN ('HC', 'MHC')
)
SELECT t.town_name, h.centre_name, s.date_basis,
       count(*) AS transaction_volume,
       round(avg(r.resale_price / f.floor_area_sqm), 2) AS avg_price_per_sqm,
```

Data Output Messages Notifications

Showing rows: 1 to 20 Page No: 1 of 1

	town_name text	centre_name text	date_basis text	transaction_volume bigint	avg_price_per_sqm numeric	avg_distance_m numeric
1	TAMPINES	BLK 137 TAMPINES STREET 11	APPROVAL	61379	2692.56	1300.3
2	YISHUN	CHONG PANG MARKET & FOOD CENTRE	APPROVAL	51795	2338.03	1225.5
3	WOODLANDS	BLKS 20/21 MARSILING LANE	APPROVAL	43443	2377.72	1635.3
4	BUKIT BATOK	BLK 254 JURONG EAST STREET 24	APPROVAL	30022	2406.48	1801.9
5	HOUGANG	KOVAN MARKET & FOOD CENTRE	APPROVAL	29359	2579.32	1601.7
6	JURONG WEST	BLKS 221 A/B BOON LAY PLACE	APPROVAL	26040	2463.97	1477.7
7	TAMPINES	BLK 137 TAMPINES STREET 11	REGISTRATION	23178	5317.30	1419.7
8	PASIR RIS	BLK 137 TAMPINES STREET 11	APPROVAL	23126	2903.96	3208.3
9	YISHUN	CHONG PANG MARKET & FOOD CENTRE	REGISTRATION	22026	4794.48	1449.6
10	WOODLANDS	BLKS 20/21 MARSILING LANE	REGISTRATION	21691	4416.18	1821.9
11	PUNGGOL	KOVAN MARKET & FOOD CENTRE	REGISTRATION	20268	5682.94	5309.8
12	SENGKANG	KOVAN MARKET & FOOD CENTRE	REGISTRATION	18860	5210.94	3724.5
13	BUKIT PANJANG	BUKIT TIMAH MARKET	APPROVAL	17114	2455.19	4631.0
14	JURONG WEST	BLKS 221 A/B BOON LAY PLACE	REGISTRATION	15326	4478.64	1447.2

Total rows: 20 Query complete 00:00:02.439 CRLF Ln 1, Col 1

Figure 9: Q6 result excerpt: the first displayed rows of the twenty reported catchment groups.

Insight. The largest returned group is Tampines assigned to Blk 137 Tampines Street 11 under the APPROVAL date basis: 61,379 sales, a mean SGD 2,692.56/m^2 and a sale-weighted mean distance of 1,300.3 m. The corresponding REGISTRATION group has 23,178 sales and a mean SGD 5,317.30/m^2. These groups represent different reporting periods, so their price difference is not an effect of the date-basis label.

A nearest-centre catchment can extend beyond one kilometre, and access can cross town boundaries. The large distances also reflect which centres are present and reliably matched in the historical catalogue: Pasir Ris sales are assigned to the Tampines centre at a mean 3,208.3 m because the NEA list has no Pasir Ris centre (Section 3.1.4). Q6 measures allocation to observed centres; it does not estimate visitor numbers or hawker demand.

3.2. Performance Measurements

3.2.1. Measurement protocol and database sizes

The measurements ran on native PostgreSQL 18.4, 64-bit Windows 11, on an Intel x86-64 system with 20 logical processors. The full version string and environment are recorded in `results/environment.json`. Each session used 32 MiB work memory, disabled JIT and disabled parallel query workers. Shared buffers were 128 MiB. The client executed one query at a time and used identical settings for baseline and index experiments.

After each load and VACUUM ANALYZE, every query was warmed once. The six queries were then executed five times each, with a reproducibly shuffled query order within each repetition. The

reported metric is PostgreSQL's server Execution Time from EXPLAIN ANALYZE with buffer reporting and per-node timing disabled. Planning time and client elapsed time are also retained, but are not mixed into the execution-time average. Result rendering is outside this metric.[11]

The baseline includes indexes required by primary and alternate keys; it excludes the four optional Task 4 indexes. Caches were not cleared between runs. These are warmed, single-client measurements, not cold-start or concurrent-load benchmarks. EXPLAIN instrumentation is present in both configurations. Five repeats describe observed variation on this machine, not a universal performance guarantee.

Database growth is measured primarily by transaction rows. The following physical sizes sum PostgreSQL's total relation size for all seven stored tables, including their required indexes and associated storage. Whole-database byte sizes are also retained in the raw measurements. The number of blocks, profiles and distance pairs changes with each increment, so growth is not simply repetition of an identical record distribution.

Stage	Newly Added Dataset	Sales	Distance pairs	Relations (MiB)
1	1990–1999	287,196	595,400	96.55
2	Mar 2012–2014	339,399	863,408	128.84
3	2015–2016	376,552	885,040	136.66
4	2017 onwards	617,120	1,020,344	186.59
5	2000–Feb 2012	986,771	1,020,968	234.04

Note: Each stage adds one HDB resale dataset to the existing database. Sales and distance pairs represent cumulative record counts.

The database grows incrementally by adding one HDB resale dataset at each stage. The increase in transaction volume is uneven because the source files contain different numbers of records. For example, Stage 2 adds 52,203 transactions, whereas Stage 5 adds 369,651 transactions. The number of distance pairs grows differently because new transactions may belong to existing HDB blocks, which already have distance records.

The stages follow the predefined loading order in the Python script rather than chronological order. Therefore, the 2000–February 2012 dataset is added at Stage 5, after the 2017 onwards dataset. All reported database sizes represent cumulative data.

There are exactly 5 sizes $\times$ 6 queries $\times$ 5 runs = 150 baseline observations in `results/baseline_measurements.csv`. The Excel workbook calculates the means directly from these raw rows and contains six editable scatter charts with numeric database-size axes. The PDF plots additionally show one sample standard deviation as error bars.

3.2.2. Runtime trends

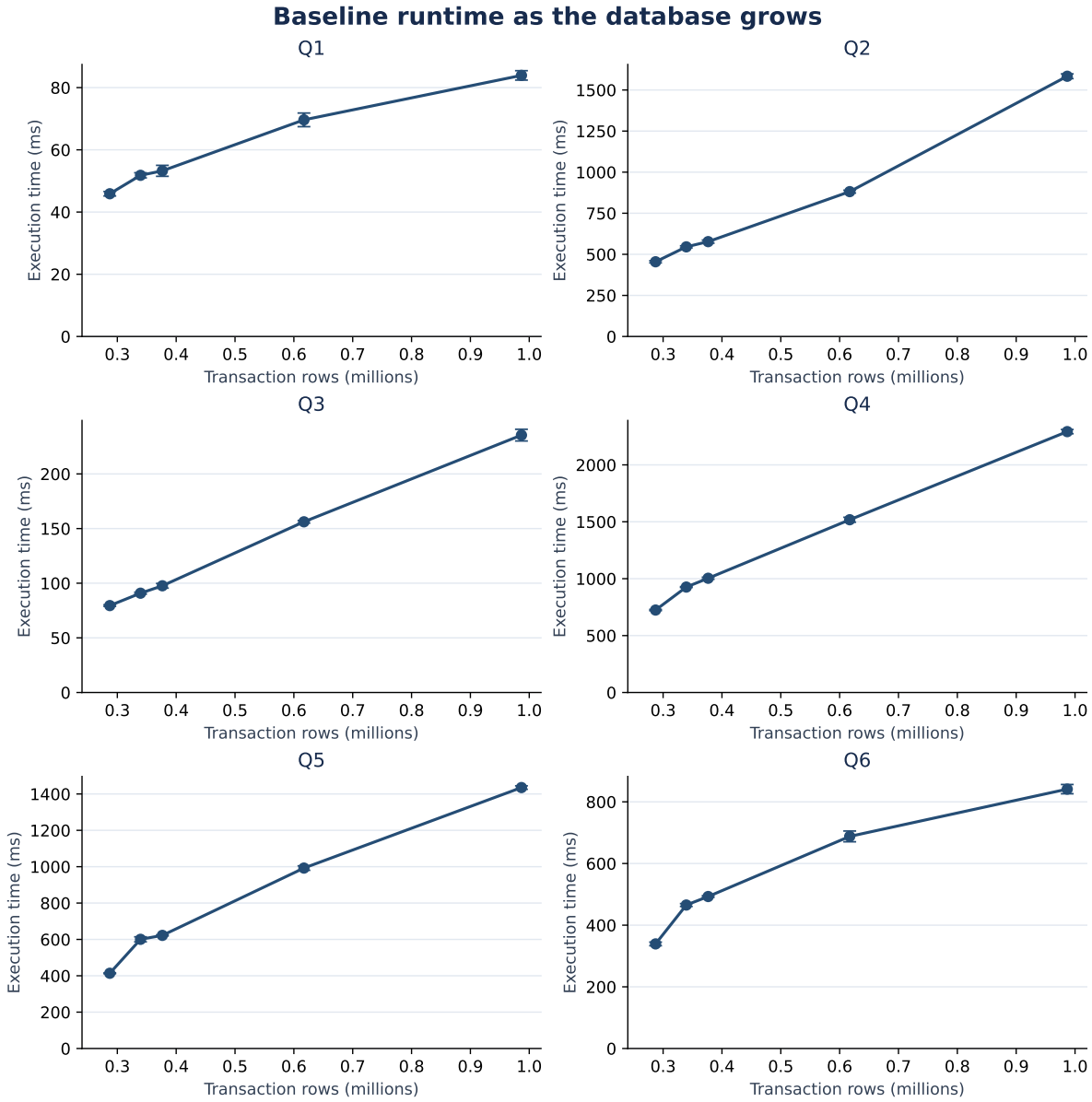


Figure 10: Database size versus baseline execution time. Each point is the mean of five runs; bars show one sample standard deviation and may be smaller than the point. Query panels use different vertical scales.

Stage	Q1	Q2	Q3	Q4	Q5	Q6
1	45.88	454.62	79.53	724.34	413.83	339.21
2	51.81	545.46	90.85	926.38	600.50	465.33
3	53.25	577.21	97.64	1,003.75	622.15	492.88
4	69.63	881.47	156.19	1,517.91	992.38	687.91
5	83.92	1,583.67	235.49	2,292.19	1,435.24	841.23

Table 5: Mean baseline execution time in milliseconds, five runs per cell.

3.2.3. Interpretation of the baseline plans

Sales increase by a factor of 3.44 from the first to the last stage. All six mean runtimes rise, but by different factors: approximately 1.83 for Q1, 3.48 for Q2, 2.96 for Q3, 3.16 for Q4, 3.47 for Q5 and 2.48 for Q6. The limited number of sizes and changing data distribution do not justify a general asymptotic complexity claim from the plots alone.

Table 6 fits a least-squares line to each query's five means (`code/performance_statistics.py`). Q2 and Q5 scale almost proportionally with sales: their runtime factors (3.48 and 3.47) match the sales factor, and Q2's intercept is close to zero. Q4 has the largest marginal cost, 218 ms per 100,000 sales. Q1 is dominated by a fixed component (33 ms intercept, 5.3 ms per 100,000 sales) because its qualifying rows never change. Q6 fits a straight line least well ($R^2 = 0.93$) because it follows the distance pairs more than the sales, as the next paragraph shows.

Query	ms per 100k sales	Intercept (ms)	R^2	Runtime factor
Q1	5.33	33.12	0.976	1.83
Q2	158.95	−20.28	0.990	3.48
Q3	22.48	14.74	0.999	2.96
Q4	217.61	158.26	0.997	3.16
Q5	139.40	85.99	0.985	3.47
Q6	66.26	219.80	0.933	2.48

Table 6: Least-squares fit of mean baseline runtime against sales rows over the five sizes; sales grow 3.44-fold.

Two growth drivers. The staged design separates them, because the stages change different relations. Stage 2 adds only 18.2% more sales but 45.0% more distance pairs: the spatial queries rise most (Q5 +45.1%, Q6 +37.2%, Q4 +27.9%), while Q1–Q3 rise 12.9–20.0%. Stage 5 adds 59.9% more sales but almost no distance pairs (+0.1%): now Q2 rises most (+79.7%), faster than sales, consistent with its growing disk sort, while Q6 rises only 22.3%. Runtime therefore depends on which relation grows, not on a single measure of database size.

Q1 grows despite fixed qualifying sales. The date window and 77,877 qualifying four/five-room transactions never change. However, the baseline full-size plan sequentially scans the transaction table and applies the date filter, then joins profile data. It retains 140,636 transactions at the date-filter step before the flat-type restriction. Extra nonqualifying source rows still have to be examined. This motivates an index beginning with transaction month.

Q2 processes the entire sales population. It must classify leases, group transactions and compute ordered medians. Its full-size plan sorts 986,771 rows and records 6,442 temporary blocks read and 6,444 written, indicating spill beyond the configured working memory. A selective date index cannot remove this full-population requirement. Its 3.48-fold increase is close to the sales-growth factor, with sorting and memory behavior also contributing.

Q3 performs two related aggregations. The baseline plan scans sales once to identify the ten towns and again for the annual maxima. Its flat-type filtering retains 23,171 profiles from 146,851. Indexing that restriction and the subsequent profile-to-sale join offers a more targeted path, particularly for the selected towns in the second aggregation.

Q4 and Q5 combine nearest-centre selection with broad sales joins. The baseline uses the distance primary key for block ordering, followed by an incremental sort on distance within each block. At full size, the eligible ranking input contains 991,517 general-hawker pairs for Q4 and

765,726 fresh-market pairs for Q5. Both queries still process the sales population after nearest-centre selection. Q4 is the slowest measured query at 2,292.19 ms.

Q6 has the same spatial ranking work but different grouping. It groups by centre, town and date basis and returns a top twenty. The final LIMIT reduces returned rows, but does not remove the earlier joins and aggregation needed to establish that ranking. Its smaller output therefore does not make runtime independent of database size.

The stage 4–5 distance population barely changes, from 1,020,344 to 1,020,968 pairs, while sales increase by 369,651. Continued growth in the spatial-query runtimes shows that the sales-side work remains material even when most block locations are already known.

4. Indices and Their Effect on Runtime

4.1. Index Creation

4.1.1. Index choices and supporting plan evidence

Four optional B+ tree indexes were selected from the observed filters, joins and ordering requirements. The CREATE INDEX statements below are those executed, as supplied in `sql/04_Create_Indexes.sql`. The following descriptions distinguish ordered key columns from included payload columns. Included columns can support an index-only scan but do not establish additional search order.[12]

CREATE INDEX statements (sql/04_Create_Indexes.sql)

```
-- Task 4. Run only AFTER collecting the five-size Task 3 baseline.
-- Baseline retains all primary-key and UNIQUE indexes.
-- These additional B+ tree indexes do not change query semantics.

-- INDEX I1 idx_resale_month_cover Q1
CREATE INDEX idx_resale_month_cover
ON housing.resale_transaction (transaction_month, flat_id) INCLUDE (resale_price);

-- INDEX I2 idx_flat_type_cover Q3
CREATE INDEX idx_flat_type_cover
ON housing.hdb_flat (flat_type, flat_id) INCLUDE (block_id, floor_area_sqm);

-- INDEX I3 idx_resale_flat_cover Q3
CREATE INDEX idx_resale_flat_cover
ON housing.resale_transaction (flat_id, transaction_month) INCLUDE (resale_price, source_file_id);

-- INDEX I4 idx_distance_order Q4
CREATE INDEX idx_distance_order
ON housing.block_hawker_distance (block_id, distance_m, hawker_id);
```

I1: transaction month followed by flat identifier. The index `idx_resale_month_cover` includes resale price as payload. Q1 can find its date interval without scanning all sales, while obtaining the profile identifier and price from the same index. In the full-size indexed plan, the transaction sequential scan is replaced by an index-only scan of the 140,636 records in the date window, with zero heap fetches. The root plan's shared hit-plus-read block count falls from 11,019 to 2,224 in the saved first-run plans, as the EXPLAIN excerpt below shows.

Q1 on the full database, EXPLAIN (ANALYZE, BUFFERS), first measured run; actual rows in parentheses (results/plans/)

```
Baseline, no optional index -- execution 84.3 ms; shared blocks hit 5,768, read 5,251
Sort  (48 rows)
  -> HashAggregate  (48)
     -> Hash Join  (77,877)
        -> Hash Join  (77,877)
           -> Hash Join  (77,877)
              -> Seq Scan on resale_transaction  (140,636)
                   Filter: transaction_month >= '1997-01-01' AND transaction_month < '2000-01-01'
              -> Hash -> Seq Scan on hdb_flat  (106,357)   Filter: flat_type IN ('4 ROOM','5 ROOM')
           -> Hash -> Seq Scan on hdb_block  (10,009)
        -> Hash -> Seq Scan on town  (27)

With the four indexes -- execution 65.4 ms; shared blocks hit 1,528, read 696
(identical tree, except that the sales scan becomes)
              -> Index Only Scan using idx_resale_month_cover on resale_transaction  (140,636)
                   Index Cond: transaction_month >= '1997-01-01' AND transaction_month < '2000-01-01'
                   Heap Fetches: 0
```

I2: flat type followed by flat identifier. The index `idx_flat_type_cover` includes block identifier and floor area. It covers the profile attributes Q3 needs after restricting to three-room flats. The indexed plan obtains 23,171 profiles using index-only scans with zero heap fetches. Q1 still uses a profile sequential scan because its two selected flat types account for a much larger part of that table. The planner is allowed to choose the cheaper scan; index use is not forced.

I3: flat identifier followed by transaction month. The index `idx_resale_flat_cover` includes resale price and source-file identifier. Its leading key supports finding sales for selected profiles, unlike I1's month-leading order. In the combined Q3 plan, the annual-maxima branch makes 9,463 index-only probes, averaging about 14.97 returned sales per probe, with zero heap fetches. The ranking branch still uses a sequential sales scan, because it examines the broad population needed to select towns. The included source identifier supports possible source-aware profile lookups; it does not imply that Q6's broad join will use this index.

I4: block identifier, distance, then hawker identifier. The index `idx_distance_order` matches the nearest-centre partition and deterministic ordering. It provides distance and both participant identifiers directly. The indexed Q4–Q6 plans use this index in place of the distance primary-key scan and incremental sort; the incremental sort disappears. These plans still read all 1,020,968 distance rows, so the index removes sorting and heap access rather than turning the whole task into one lookup per block.

The existing primary-key and alternate-key indexes are retained in all configurations. No index is created simply on an already unique primary key. Index-only scans benefit from the visibility maps produced after the bulk load; a heavily updated operational database may require heap visits and may show different performance.

4.2. Effect on Runtime

4.2.1. Isolated index experiments

Each optional index was first created alone on the full database, with the other three absent. Its target query was warmed once, measured five times, and compared with that query's full-size Task 3 baseline. The index was then dropped before the next candidate was tested. This gives 20 isolated-index observations and avoids attributing the combined configuration's gain to every individual index.

Index	Query	Baseline	Alone	Reduction	MiB	p
I1	Q1	83.92	58.74	30.00%	38.19	0.008
I2	Q3	235.49	227.77	3.28%	7.10	0.079
I3	Q3	235.49	181.15	23.08%	46.69	0.008
I4	Q4	2,292.19	2,073.73	9.53%	39.51	0.008

Table 7: Index tested alone: mean execution time in milliseconds, $n = 5$; storage in MiB; p from an exact two-sided permutation test against the baseline runs.

I1 and I3 deliver the clearest isolated reductions. I2's 7.72 ms difference is modest relative to the observed standard deviations of 5.35 ms for baseline Q3 and 5.97 ms with I2 alone. Its plan improvement is visible, but five trials do not establish a robust standalone benefit across machines or workloads. It is retained here to evaluate its contribution alongside I3. I4 improves Q4 by 218.46 ms, consistent with removing the incremental sort and using an index-only distance scan.

Measured creation times were 0.47, 0.14, 0.54 and 0.49 seconds for I1–I4 respectively, about 1.64 seconds in total. The four indexes occupy 131.48 MiB, approximately 56.18% of the baseline seven-table footprint of 234.04 MiB. This storage cost is material. Future inserts and updates would also have to maintain the indexes; that write cost was not measured because the experiment indexes the completed load.

4.2.2. Combined index experiment

All four indexes were then created together. Each of the six queries was warmed once and measured five times under the same settings, producing 30 further observations. Their result CSVs were byte-identical to the full-size baseline outputs. Runtime reduction is $100(1 - T_{\text{indexed}}/T_{\text{baseline}})$; speedup is $T_{\text{baseline}}/T_{\text{indexed}}$. These are comparisons with the same 986,771-sale database, not with the smaller initial instance.

The measurements are preserved separately in `results/index_individual_measurements.csv` and `results/indexed_measurements.csv`. Together with the baseline, the package contains 200 measured executions. Warm-up executions are excluded from these counts.

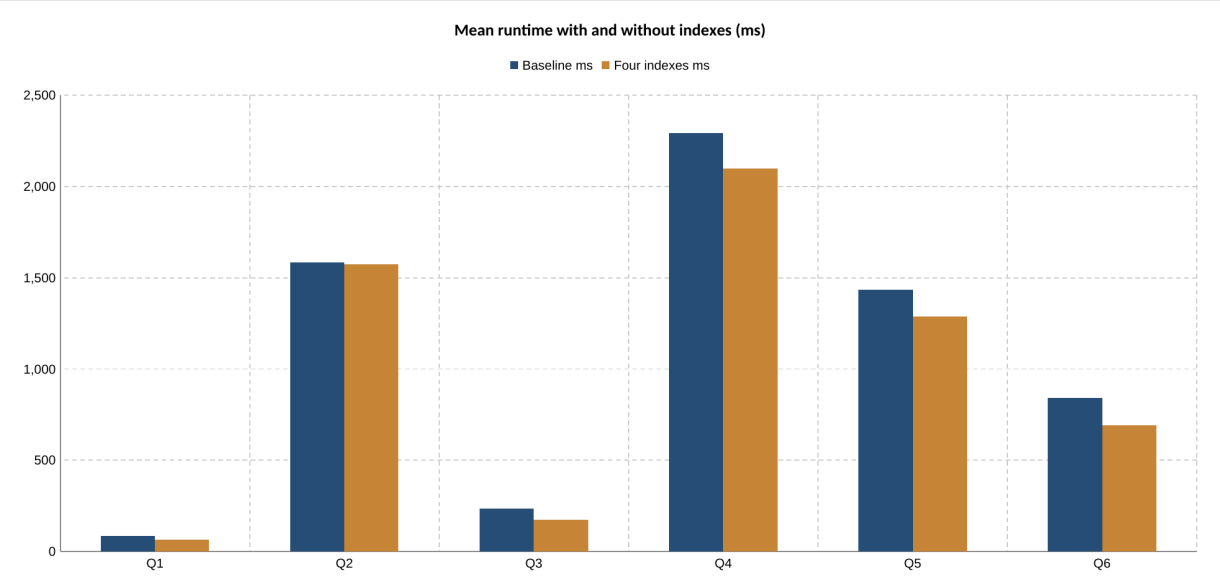


Figure 11: Excel bar chart from the Index Comparison sheet of `Performance_Measurements.xlsx`: full-size mean runtimes without and with the four optional indexes, $n = 5$ each. Means are calculated in the workbook from the raw measurements; standard deviations are in Table 8.

Query	Baseline mean ± SD	Indexed mean ± SD	Reduction	Speedup	p
Q1	83.92 ± 1.51	63.43 ± 2.39	24.42%	1.32×	0.008
Q2	1,583.67 ± 14.40	1,573.78 ± 11.50	0.62%	1.01×	0.262
Q3	235.49 ± 5.35	174.25 ± 2.33	26.01%	1.35×	0.008
Q4	2,292.19 ± 18.69	2,098.47 ± 5.92	8.45%	1.09×	0.008
Q5	1,435.24 ± 9.51	1,288.93 ± 11.65	10.19%	1.11×	0.008
Q6	841.23 ± 15.01	690.18 ± 6.62	17.96%	1.22×	0.008

Table 8: Full-size execution times in milliseconds. SD is the sample standard deviation of five runs; p from an exact two-sided permutation test.

Q1 and Q3 show the largest percentage reductions, matching their selective access opportunities. Q4–Q6 also improve after their spatial sort is removed, but still perform broad distance and sales processing. Q4 remains the slowest query at 2,098.47 ms: indexing helps without eliminating its principal amount of work.

Q2 changes by only 0.62%, less than its observed run-to-run variation. Its saved plan still uses sequential scans and the same median-related sort, including 6,442 temporary blocks read and 6,444 written. There is no convincing index benefit for Q2 in this experiment. A separate memory-tuning or query-design experiment would be needed to investigate this bottleneck; those changes were not mixed into the index comparison.

Statistical check. With only five runs per configuration, each difference is tested with an exact two-sided permutation test over all 252 ways of splitting the ten timings (`code/performance_statistics.py`). It needs no distributional assumption, and its smallest attainable value is $p = 2/252 \approx 0.008$. Every combined and isolated comparison reaches that minimum, which means all five indexed runs were faster than all five baseline runs, with two exceptions: Q2 with all indexes ($p = 0.26$) and I2 alone on Q3 ($p = 0.079$). These are exactly the two cases the discussion treats as no convincing improvement. Because baseline and indexed runs were not interleaved, the test quantifies run-to-run variation, not systematic drift between the two sessions.

4.2.3. Trade-offs and limits of the comparison

Index benefits are not additive. I1 alone gives a lower Q1 mean than the four-index configuration, even though both plans use I1. The saved isolated plan has 2,224 shared-buffer hits and no reads, while the combined plan has 1,528 hits and 696 reads. The different buffer residency is consistent with part of this difference; the runs do not isolate cache effects sufficiently to attribute the entire change to one cause. Creating more indexes does not guarantee a faster instance of every query.

The combined configuration retains all four candidates to make the measured setup explicit. For a narrower workload, I2 would be the first candidate to reassess because its isolated gain is small, although it participates in the faster combined Q3 plan. I1 is justified when the date-window analysis is frequent; I3 benefits profile-based sales retrieval; I4 supports repeated nearest-centre work. Their value should be balanced against the measured 131.48 MiB storage cost and unmeasured write-maintenance cost.

Index	Serves	Alone	p	MiB	Decision and reason
I1	Q1	−30.0%	0.008	38.2	Keep: replaces the full sales scan with an index-only date-range scan
I2	Q3	−3.3%	0.079	7.1	Reassess: small and not significant alone; cheap to keep while it takes part in the faster combined Q3 plan
I3	Q3	−23.1%	0.008	46.7	Keep: index-only probes for the selected towns' three-room sales
I4	Q4, Q5, Q6	−9.5%	0.008	39.5	Keep: removes the incremental sort in all three spatial queries; largest absolute saving (218 ms per Q4 run)
None	Q2	—	—	—	No index helps: the bottleneck is a full-population sort that spills to disk, which needs a memory or query-design change

Table 9: Evidence and decision for each optional index on the full database.

The baseline always precedes the index experiments, so background activity and cache evolution can affect small differences. Query order is shuffled within repetitions, but configurations are not alternated randomly. The recorded plans, buffer counters and standard deviations support interpretation of this run; they do not replace a larger controlled benchmark. The most defensible conclusions are the clear changes in access paths and the larger reductions observed for Q1 and Q3, together with the absence of a material improvement for Q2.

The SQL reproduced in this report is identical to the executable files supplied with it. The accompanying workbook, raw measurements, complete result CSVs and JSON plans allow the calculations and interpretations to be checked without relying on screenshots alone.

Declaration of AI Use

In accordance with the project guidelines, we declare that we used AI tools, including ChatGPT and Claude, to assist with language polishing and code debugging. Our group takes full responsibility for the content, code, and results submitted in this project.

We found these tools useful for improving writing clarity and identifying coding errors. However, developing ideas and verifying technical content still required our own judgment. AI-generated suggestions did not always align with the lecture material or use the correct notation. We therefore reviewed these suggestions against the course content and made manual corrections where necessary.

References

[1] M. Herschel. *Data Systems Project*, AY26S1. Supplied assignment brief, Tasks 1–4, Appendix A and project logistics.

[2] M. Herschel. *Chapter 3 Entity Relationship Model*. Supplied lecture `03_w3_ER.pdf`, PDF pp. 17–20, 32–37 and 40–44.

[3] M. Herschel. *Chapter 4 Relational Model*. Supplied lecture `04_w4_relational_model.pdf`, PDF pp. 47 and 52–62.

[4] M. Herschel. *Chapter 5 Normalization*. Supplied lecture `05_w5_normalization.pdf`, PDF pp. 42, 47 and 53–58. Computed attributes are discussed on PDF p. 58, printed slide 65.

[5] Housing & Development Board. Resale Flat Prices collection. Frozen files retrieved on 17 September 2026 (Singapore time).

[6] National Environment Agency. List of Government Markets Hawker Centres. Catalogue snapshot inspected on 16 September 2026. Centre coordinates are enriched from the supplied `hawker_geo.csv`.

[7] Lan et al. Land and Transport Singapore dataset, HDB geolocation file. Associated publication: Site Selection via Learning Graph Convolutional Neural Networks, *Remote Sensing* 14(15), 3579, 2022.

[8] PostgreSQL documentation. Constraints and CREATE DATABASE.

[9] data.gov.sg. Download dataset API and dataset metadata API. Source-specific metadata and hashes accompany the frozen raw files.

[10] PostgreSQL 18 documentation. COPY.

[11] PostgreSQL 18 documentation. Using EXPLAIN.

[12] PostgreSQL 18 documentation. Index-only scans and covering indexes.

A. Schema Creation Commands

The complete contents of `sql/01_Create_Database.sql`, as executed. The psql meta-commands (`\set`, `\connect`) are omitted when the same statements are run from pgAdmin, as the file header explains.

sql/01_Create_Database.sql

```
-- SD6103 Assignment 1 Task 1
-- Lecture mapping: entity keys become primary keys; mandatory 1:N links
-- become NOT NULL foreign keys; M:N Proximity becomes a pair relation.
-- Chapter 5 computed-attribute rule: proximity_class is exposed by a view.
-- Complete setup script for psql. PostgreSQL 14 or later; no extensions.
-- Run from the package root: psql -U postgres -d postgres -f sql/01_Create_Database.sql
-- Requires a running PostgreSQL server and a role with CREATEDB permission.
-- This script creates a new database and does not delete existing objects.
-- If the database exists, inspect it before reusing the schema section.
-- In pgAdmin, execute CREATE DATABASE alone, reconnect to the new database,
-- and then execute the BEGIN-to-COMMIT section; omit the backslash commands.
\set ON_ERROR_STOP on

CREATE DATABASE sd6103_hdb_hawker
    WITH ENCODING = 'UTF8' TEMPLATE = template0;

\connect sd6103_hdb_hawker

BEGIN;

CREATE SCHEMA housing;

CREATE TABLE housing.town (
    town_id       bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    town_name     text NOT NULL UNIQUE,
    CONSTRAINT town_name_canonical CHECK (
        town_name <> '' AND town_name = upper(btrim(town_name))
    )
);

CREATE TABLE housing.hdb_block (
    block_id      bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    town_id       bigint NOT NULL REFERENCES housing.town(town_id)
                  ON UPDATE RESTRICT ON DELETE RESTRICT,
    block_no      text NOT NULL,
    street_name   text NOT NULL,
    latitude      double precision,
    longitude     double precision,
    CONSTRAINT block_address_uk UNIQUE (block_no, street_name),
    CONSTRAINT block_no_canonical CHECK (
        block_no <> '' AND block_no = upper(btrim(block_no))
    ),
    CONSTRAINT street_name_canonical CHECK (
        street_name <> '' AND street_name = upper(btrim(street_name))
    ),
    CONSTRAINT block_coordinates_paired CHECK (
        (latitude IS NULL) = (longitude IS NULL)
    ),
    CONSTRAINT block_latitude_valid CHECK (latitude BETWEEN -90 AND 90),
    CONSTRAINT block_longitude_valid CHECK (longitude BETWEEN -180 AND 180)
);

CREATE TABLE housing.hdb_flat (
    flat_id              bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    block_id             bigint NOT NULL REFERENCES housing.hdb_block(block_id)
                         ON UPDATE RESTRICT ON DELETE RESTRICT,
    flat_type            text NOT NULL,
    flat_model           text NOT NULL,
    floor_area_sqm       numeric(8,2) NOT NULL,
    storey_lower         smallint NOT NULL,
    storey_upper         smallint NOT NULL,
```

```
    lease_commence_year  smallint NOT NULL,
    CONSTRAINT flat_profile_uk UNIQUE (
        block_id, flat_type, flat_model, floor_area_sqm,
        storey_lower, storey_upper, lease_commence_year
    ),
    CONSTRAINT flat_type_valid CHECK (flat_type IN (
        '1 ROOM', '2 ROOM', '3 ROOM', '4 ROOM', '5 ROOM',
        'EXECUTIVE', 'MULTI-GENERATION'
    )),
    CONSTRAINT flat_model_canonical CHECK (
        flat_model <> '' AND flat_model = upper(btrim(flat_model))
    ),
    CONSTRAINT floor_area_positive_finite CHECK (
        floor_area_sqm > 0 AND floor_area_sqm < 'Infinity'::numeric
    ),
    CONSTRAINT storey_range_valid CHECK (
        storey_lower >= 1 AND storey_upper >= storey_lower
    ),
    CONSTRAINT lease_year_valid CHECK (lease_commence_year BETWEEN 1900 AND 9999)
);

COMMENT ON TABLE housing.hdb_flat IS
'An observed flat-attribute profile, not an identified physical unit. Multiple units may share one profile.';

CREATE TABLE housing.source_file (
    source_file_id   bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    file_name        text NOT NULL,
    sha256           text NOT NULL UNIQUE,
    date_basis       text NOT NULL,
    CONSTRAINT source_filename_nonblank CHECK (
        file_name <> '' AND file_name = btrim(file_name)
    ),
    CONSTRAINT source_sha256_valid CHECK (sha256 ~ '^[0-9a-f]{64}$'),
    CONSTRAINT source_date_basis_valid CHECK (
        date_basis IN ('APPROVAL', 'REGISTRATION')
    )
);

CREATE TABLE housing.resale_transaction (
    transaction_id                  bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    flat_id                         bigint NOT NULL REFERENCES housing.hdb_flat(flat_id)
                                    ON UPDATE RESTRICT ON DELETE RESTRICT,
    source_file_id                  bigint NOT NULL REFERENCES housing.source_file(source_file_id)
                                    ON UPDATE RESTRICT ON DELETE RESTRICT,
    source_row_number               bigint NOT NULL,
    transaction_month               date NOT NULL,
    resale_price                    numeric(12,2) NOT NULL,
    remaining_lease_months_reported  smallint,
    remaining_lease_precision       text,
    CONSTRAINT source_row_uk UNIQUE (source_file_id, source_row_number),
    CONSTRAINT source_row_positive CHECK (source_row_number >= 1),
    CONSTRAINT transaction_month_valid CHECK (
        transaction_month >= DATE '1900-01-01'
        AND transaction_month < DATE '10000-01-01'
        AND EXTRACT(DAY FROM transaction_month) = 1
    ),
    CONSTRAINT resale_price_positive_finite CHECK (
        resale_price > 0 AND resale_price < 'Infinity'::numeric
    ),
    CONSTRAINT reported_lease_range CHECK (
        remaining_lease_months_reported BETWEEN 0 AND 1188
    ),
    CONSTRAINT reported_lease_precision_valid CHECK (
        remaining_lease_precision IN ('MONTH', 'YEAR')
    ),
    CONSTRAINT reported_lease_fields_paired CHECK (
        (remaining_lease_months_reported IS NULL)
        = (remaining_lease_precision IS NULL)
    ),
    CONSTRAINT reported_years_are_whole CHECK (
        remaining_lease_precision IS DISTINCT FROM 'YEAR'
```

```
        OR remaining_lease_months_reported % 12 = 0
    )
);

COMMENT ON COLUMN housing.resale_transaction.transaction_month IS
'First day encodes a source month, not the exact day of sale. Meaning is given by source_file.date_basis.';
COMMENT ON COLUMN housing.resale_transaction.source_row_number IS
'1-based data record number in the immutable source file, excluding its header.';

CREATE TABLE housing.hawker_centre (
    hawker_id             bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    centre_name           text NOT NULL UNIQUE,
    address               text NOT NULL,
    centre_type           text NOT NULL,
    cooked_food_stalls    integer NOT NULL,
    market_produce_stalls integer NOT NULL,
    latitude              double precision,
    longitude             double precision,
    CONSTRAINT centre_name_canonical CHECK (
        centre_name <> '' AND centre_name = upper(btrim(centre_name))
    ),
    CONSTRAINT centre_address_nonblank CHECK (
        address <> '' AND address = btrim(address)
    ),
    CONSTRAINT centre_type_valid CHECK (centre_type IN ('HC', 'MHC', 'MK')),
    CONSTRAINT cooked_stalls_nonnegative CHECK (cooked_food_stalls >= 0),
    CONSTRAINT market_stalls_nonnegative CHECK (market_produce_stalls >= 0),
    CONSTRAINT hawker_coordinates_paired CHECK (
        (latitude IS NULL) = (longitude IS NULL)
    ),
    CONSTRAINT hawker_latitude_valid CHECK (latitude BETWEEN -90 AND 90),
    CONSTRAINT hawker_longitude_valid CHECK (longitude BETWEEN -180 AND 180)
);

CREATE TABLE housing.block_hawker_distance (
    block_id     bigint NOT NULL REFERENCES housing.hdb_block(block_id)
                 ON UPDATE RESTRICT ON DELETE RESTRICT,
    hawker_id    bigint NOT NULL REFERENCES housing.hawker_centre(hawker_id)
                 ON UPDATE RESTRICT ON DELETE RESTRICT,
    distance_m   double precision NOT NULL,
    PRIMARY KEY (block_id, hawker_id),
    -- pi * 6,371,000 metres is the maximum great-circle distance.
    -- The slightly larger bound also rejects Infinity and NaN.
    CONSTRAINT distance_valid CHECK (distance_m BETWEEN 0 AND 20040000)
);

COMMENT ON TABLE housing.block_hawker_distance IS
'One Haversine distance per geocoded block-centre pair. ETL must validate coordinates, coverage and
   recomputation after coordinate changes.';

CREATE FUNCTION housing.proximity_class(distance_m double precision)
RETURNS text
LANGUAGE sql
IMMUTABLE
STRICT
PARALLEL SAFE
AS $function$
    SELECT CASE
        WHEN distance_m >= 0 AND distance_m < 500 THEN 'NEAR'
        WHEN distance_m >= 500 AND distance_m < 1000 THEN 'MEDIUM'
        WHEN distance_m >= 1000 AND distance_m <= 20040000 THEN 'FAR'
        ELSE NULL
    END;
$function$;

-- Distance determines class, so class is derived rather than duplicated
-- as a non-key stored attribute in block_hawker_distance.
CREATE VIEW housing.v_block_hawker_proximity AS
SELECT block_id, hawker_id, distance_m,
       housing.proximity_class(distance_m) AS proximity_class
FROM housing.block_hawker_distance;
```

```sql
-- Q4: exactly one eligible nearest centre per block with distance data.
-- MK means market-only and is not in the eligible hawker population.
CREATE VIEW housing.v_nearest_hawker AS
SELECT block_id, hawker_id, distance_m,
       housing.proximity_class(distance_m) AS proximity_class
FROM (
    SELECT d.*,
           ROW_NUMBER() OVER (
               PARTITION BY d.block_id ORDER BY d.distance_m, d.hawker_id
           ) AS nearest_rank
    FROM housing.block_hawker_distance AS d
    JOIN housing.hawker_centre AS h USING (hawker_id)
    WHERE h.centre_type IN ('HC', 'MHC')
) AS ranked
WHERE nearest_rank = 1;

-- Q5: filter to fresh-market hawkers BEFORE selecting the nearest one.
CREATE VIEW housing.v_nearest_market_hawker AS
SELECT block_id, hawker_id, distance_m,
       housing.proximity_class(distance_m) AS proximity_class
FROM (
    SELECT d.*,
           ROW_NUMBER() OVER (
               PARTITION BY d.block_id ORDER BY d.distance_m, d.hawker_id
           ) AS nearest_rank
    FROM housing.block_hawker_distance AS d
    JOIN housing.hawker_centre AS h USING (hawker_id)
    WHERE h.centre_type IN ('HC', 'MHC')
      AND h.market_produce_stalls > 0
) AS ranked
WHERE nearest_rank = 1;

-- Q1-Q3 support: one row per sale. Values derived across tables are views,
-- not generated columns that would need to reference another table.
CREATE VIEW housing.v_resale_analysis AS
SELECT r.transaction_id, r.source_file_id, r.source_row_number,
       r.flat_id, f.block_id, b.town_id, t.town_name,
       f.flat_type, f.flat_model, f.floor_area_sqm,
       f.storey_lower, f.storey_upper, f.lease_commence_year,
       r.transaction_month,
       EXTRACT(YEAR FROM r.transaction_month)::integer AS transaction_year,
       r.resale_price,
       r.resale_price / f.floor_area_sqm AS price_per_sqm,
       r.remaining_lease_months_reported,
       r.remaining_lease_precision,
       CASE
           WHEN r.remaining_lease_months_reported IS NOT NULL
               THEN r.remaining_lease_months_reported::integer
           WHEN estimate.months BETWEEN 0 AND 1188 THEN estimate.months
           ELSE NULL
       END AS remaining_lease_months,
       CASE
           WHEN r.remaining_lease_precision = 'MONTH' THEN 'REPORTED_MONTH'
           WHEN r.remaining_lease_precision = 'YEAR' THEN 'REPORTED_YEAR'
           WHEN estimate.months BETWEEN 0 AND 1188 THEN 'ESTIMATED_YEAR'
           ELSE 'UNAVAILABLE'
       END AS remaining_lease_basis
FROM housing.resale_transaction AS r
JOIN housing.hdb_flat AS f USING (flat_id)
JOIN housing.hdb_block AS b USING (block_id)
JOIN housing.town AS t USING (town_id)
CROSS JOIN LATERAL (
    SELECT (99 - (
        EXTRACT(YEAR FROM r.transaction_month)::integer - f.lease_commence_year
    )) * 12 AS months
) AS estimate;

-- Only key/uniqueness indexes are created automatically here.
-- Leave optional performance indexes for Task 4.
COMMIT;
```